

Laboratory Record

submitted as part of

Programming for Data Science Lab

LAB ASSESSMENT-8

by



Reg No: 26MML0031
Name: Hasnain H. Makada

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SEPTEMBER 2026

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record

INDEX

Week No	Date	Sl. No.	Type	Topic	Title with download Link for document	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling	14
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. Normalize rating values.	18

3	23-07-2026	5	PP	NumPy	Creating and Manipulating Arrays Using NumPy	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape.	30
3	23-07-2026	6	EP	NumPy	Warehouse Shipment and Logistic Analytics Using Numpy	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra	35
4	30/7/2026	7	PP	Pandas, Numpy, Matplotlib	Feature Engineering and Exploratory Data Analysis	Create a DataFrame with columns Feature1, Feature2, and Target containing random data. Perform the following operations: create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (low, medium, high), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 vs. Feature2 colored by Target, and interpret any visible patterns.	46

4	30/7/2026	8	EP	Pandas, NumPy, Matplotlib	Employee Attrition Prediction Using Workforce Indicators	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing: OvertimeHours JobSatisfactionScore SalaryLevel TrainingHours WorkLifeBalance YearsAtCompany PromotionCount AttritionStatus Perform: 1. Create Log_OvertimeHours. 2. Categorize JobSatisfactionScore into: - o Disengaged - o Satisfied - o Highly Engaged 3. Generate correlation matrix. 4. Create a Treemap Visualization of employee categories. Analyze workforce trends.	54
5	06/08/2026	9	PP	Web Scraping	requests, BeautifulSoup, pandas, json	Scrape data from a webpage and store it in a structured format like CSV or JSON	63
5	06/08/2026	10	EP	Web Scraping	Online Job Market Analytics	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.	70
6	20/08/2026	11	PP	Plotly	Interactive Bar Chart with Plotly	Create an interactive bar chart showing the population of different cities.	80
6	20/08/2026	12	PP	Plotly	Interactive Scatter Plot with Plotly	Create an interactive scatter plot showing the relationship between house size and price.	88

7	27/08/2026	13	PP	Bokeh	Interactive Scatter Plot with Bokeh	Create an interactive scatter plot showing the relationship between petal length and petal width from the Iris dataset using Bokeh.	101
7	27/08/2026	14	EP	Bokeh	Interactive Line Plot with Bokeh	Create an interactive line plot showing the daily temperatures over a week using Bokeh.	105
7	27/08/2026	15	EP	Bokeh	Interactive Bar Chart with Bokeh	Create an interactive bar chart showing the sales figures of different products using Bokeh.	109
8	03/09/2026	16	PP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly.	113
8	03/09/2026	17	EP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly.	122

PP: Practice Program; EP: Exercise Program; CP: Challenging Program

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Title	Web Scraping and Data Collection			
Problem Statement	Scrape data from a webpage and store it in a structured format like CSV or JSON format			
Objectives	• To understand the concept of web scraping and data collection. • To retrieve webpage content using the Requests library. • To extract required information from HTML using BeautifulSoup. • To organize the scraped data using Pandas DataFrame. • To store the collected data in structured formats such as CSV and JSON.			
Dataset URL Link / File	https://quotes.toscrape.com			
Function Details	Sl. No.	Function	Package	Description of Function
	1	requests.get()	requests	Retrieves the HTML content of a webpage
	2	BeautifulSoup()	bs4	Parses the HTML content of the webpage
	3	find_all()	BeautifulSoup	Finds all matching HTML elements
	4	find()	BeautifulSoup	Finds the first matching HTML element
	5	DataFrame()	pandas	Converts collected data into tabular format
	6	to_csv()	pandas	Stores data in CSV format
	7	json.dump()	json	Stores Python data in JSON format
	8	json.load()	json	Reads data from a JSON file
Steps/ Process/ Procedure/	1. Import the required libraries: requests, BeautifulSoup, pandas and json. 2. Specify the URL of the webpage to be scraped.			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Algorithm/ Flowchart/ Model	3. Send an HTTP GET request using requests.get(). 4. Parse the HTML content using BeautifulSoup. 5. Locate the required HTML elements containing quotes, authors and tags. 6. Extract the required data from the webpage. 7. Store the extracted data in a Python list. 8. Convert the collected data into a Pandas DataFrame. 9. Save the structured data in CSV format using to_csv(). 10. Save the same data in JSON format using json.dump(). 11. Display the collected data to verify the output.
Program	<pre> import requests from bs4 import BeautifulSoup import pandas as pd import json url = "https://quotes.toscrape.com/" response = requests.get(url) print("Status Code:", response.status_code) soup = BeautifulSoup(response.text, "html.parser") print("Website Title:", soup.title.text) quotes = [] for item in soup.find_all("div", class_="quote"): quote = item.find("span", class_="text").text author = item.find("small", class_="author").text tags = [tag.text for tag in item.find_all("a", class_="tag")] quotes.append({ "Quote": quote, "Author": author, "Tags": " ".join(tags) }) print("Number of quotes collected:", len(quotes)) </pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

*Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore*

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
df = pd.DataFrame(quotes)

df

df.to_csv("quotes.csv", index=False)

print("Data saved successfully in CSV format.")

with open("quotes.json", "w", encoding="utf-8") as file:
    json.dump(quotes, file, indent=4, ensure_ascii=False)

print("Data saved successfully in JSON format.")

csv_data = pd.read_csv("quotes.csv")

csv_data

all_quotes = []

page = 1

while True:

    url = f"https://quotes.toscrape.com/page/{page}/"

    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")

    items = soup.find_all("div", class_="quote")

    if not items:
        break

    for item in items:

        quote = item.find("span", class_="text").text
        author = item.find("small", class_="author").text

        tags = [
            tag.text
            for tag in item.find_all("a", class_="tag")
        ]

        all_quotes.append({
            "Quote": quote,
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

*Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore*

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

	<pre> "Author": author, "Tags": ", ".join(tags) }) page += 1 print("Total quotes collected:", len(all_quotes)) df = pd.DataFrame(all_quotes) df df.to_csv("quotes_dataset.csv", index=False) with open("quotes_dataset.json", "w", encoding="utf-8") as file: json.dump(all_quotes, file, indent=4, ensure_ascii=False) print("CSV and JSON files created successfully.") </pre>
Output Snapshots With Explanation/ Observations	The required libraries requests, BeautifulSoup, pandas and json are imported for web scraping, HTML parsing, data organization and structured data storage. <pre> 1 import requests 2 from bs4 import BeautifulSoup 3 import pandas as pd 4 import json </pre> <pre> 1 url = "https://quotes.toscrape.com/" 2 3 response = requests.get(url) 4 5 print("Status Code:", response.status_code) </pre> Status Code: 200 <i>Fig. 1.1: Importing Required Python Libraries</i>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

The webpage <https://quotes.toscrape.com/> is accessed using `requests.get()`. The status code 200 confirms that the webpage was successfully retrieved.

```
[7]: 1 quotes = []
      2
      3 for item in soup.find_all("div", class_="quote"):
      4
      5     quote = item.find("span", class_="text").text
      6     author = item.find("small", class_="author").text
      7
      8     tags = [
      9         tag.text
     10         for tag in item.find_all("a", class_="tag")
     11     ]
     12
     13 quotes.append({
     14     "Quote": quote,
     15     "Author": author,
     16     "Tags": ", ".join(tags)
     17 })
     18
     19 print("Number of quotes collected:", len(quotes))

Number of quotes collected: 10
```

Fig. 1.2: Accessing the Webpage

The HTML content of the webpage is parsed using BeautifulSoup. The webpage title confirms that the correct page has been successfully loaded.

Student Reg. No: 26MML0031 Name: Hasnain Makada

*Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore*

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
number of quotes collected: 10

[8]: 1 df = pd.DataFrame(quotes)
      2
      3 df

[8]:
```

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig. 1.3: Parsing the HTML and displaying the dataset Content

```
1 df.to_csv("quotes.csv", index=False)
2
3 print("Data saved successfully in CSV format.")

Data saved successfully in CSV format.
```

Fig. 1.4: Saving Data in CSV Format

The scraped data is successfully stored in CSV format using the pandas `to_csv()` function.

The scraped data is successfully stored in JSON format using the `json.dump()` function.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
1 with open("quotes.json", "w", encoding="utf-8") as file:
2     json.dump(quotes, file, indent=4, ensure_ascii=False)
3
4 print("Data saved successfully in JSON format.")
```

Data saved successfully in JSON format.

Fig. 1.5: Saving Data in JSON Format

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig 1.6: Displaying the saved json and csv

The scraped data is successfully stored in JSON format using the `json.dump()` function.

The saved CSV file is read using `pandas.read_csv()` and displayed to verify that the data has been stored correctly.

The saved JSON file is read using `json.load()` and the stored records are displayed successfully, confirming correct JSON storage.

Conclusions

The experiment was successfully completed by scraping data from the given webpage using Python. The webpage content was retrieved using the requests library and parsed using BeautifulSoup. The required information such as quotes, authors, and tags was extracted and organized using a Pandas DataFrame. Finally, the collected data was successfully stored in structured CSV and JSON formats for further analysis and use.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data
2. Python Program: [GitHub link](#)
3. Class lecture: [Lecture link](#)

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

Title	Online Job Market Analytics			
Problem Statement	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.			
Objectives	• To understand web scraping from a dynamic job portal using Selenium. • To extract job title, company name, location and salary information from job listings. • To organize and clean the collected data using a Pandas DataFrame. • To save the structured job dataset in CSV format. • To perform location-wise analysis and visualize job distribution using Matplotlib.			
Dataset URL Link / File	Job Portal: https://www.naukri.com Search role used: Data Scientist Cities: Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi			
Functions Details	Sl. No.	Function	Package	Description of function
	1	Firefox()	selenium	Starts a Firefox browser session controlled by Selenium.
	2	driver.get()	selenium	Opens the specified job-portal URL in the automated browser.
	3	execute_script()	selenium	Executes JavaScript to scroll the dynamically loaded webpage.
	4	find_elements()	selenium	Finds all matching job-card elements on the page.
	5	find_element()	selenium	Finds a required field inside an individual job card.
	6	time.sleep()	time	Pauses execution briefly to allow dynamic content to load.
	7	DataFrame()	pandas	Converts the collected job records into tabular form.
	8	drop_duplicates()	pandas	Removes duplicate job records from the DataFrame.
	9	reset_index()	pandas	Resets row indexing after data cleaning.
	10	to_csv()	pandas	Saves the structured job dataset as a CSV file.
	11	value_counts()	pandas	Counts job records for each location for analysis.
	12	bar()/savefig()	matplotlib	Creates and saves the location-wise job-demand bar chart.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Selenium, Pandas, Matplotlib and supporting Python modules. 2. Define the job role, location and maximum number of job listings to collect. 3. Launch the browser using Selenium WebDriver and construct the Naukri search URL. 4. Open the dynamic job-search page and allow sufficient time for the page to load. 5. Scroll the webpage so that dynamically loaded job cards become available. 6. Locate job-card elements using CSS selectors. 7. Extract Job Title, Company Name, Location and Salary from each job card. 8. Store the extracted records in a Python list and convert them into a Pandas DataFrame. 9. Remove duplicate records and save the cleaned dataset as a CSV file. 10. Repeat the search for Bangalore, Hyderabad, Pune, Chennai, Mumbai			

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

	and Delhi. 11. Use value_counts() to prepare location-wise job analysis and create a bar chart. 12. Save the visualization and multi-city CSV file, display the final summary and close the browser. START -> Open Job Portal -> Scrape Listings -> DataFrame -> CSV -> Location Analysis -> Bar Chart -> STOP
Program	<pre> import time import pandas as pd import matplotlib.pyplot as plt from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.firefox.options import Options from selenium.common.exceptions import NoSuchElementException JOB_ROLE = "data scientist" LOCATION = "bangalore" MAX_JOBS = 30 options = Options() driver = webdriver.Firefox(options=options) driver.set_window_size(1400, 900) def make_slug(text): return "-".join(text.lower().strip().split()) def get_text(card, selectors, default="Not Available"): for selector in selectors: try: element = card.find_element(By.CSS_SELECTOR, selector) text = element.text.strip() if text: return text except NoSuchElementException: pass return default job_slug = make_slug(JOB_ROLE) location_slug = make_slug(LOCATION) url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}" print("Search URL:", url) driver.get(url) time.sleep(5) print("Page Title:", driver.title) print("Current URL:", driver.current_url) </pre>

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
for i in range(3):
    driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);")
    time.sleep(2)
driver.execute_script("window.scrollTo(0, 0);")

cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-tuple")
print("Number of job cards found:", len(cards))

jobs = []
for card in cards[:MAX_JOBS]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    location = get_text(card, ["span.locWdth", ".locWdth",
"[class*='location']"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
default="Not Disclosed")
    jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": location,
        "Salary": salary
    })

print("Jobs scraped:", len(jobs))
df = pd.DataFrame(jobs)
df = df.drop_duplicates().reset_index(drop=True)
print("Total unique jobs:", len(df))
print("Dataset Shape:", df.shape)
print(df.head(10))
print("Salary Information:")
print(df["Salary"].value_counts())

df.to_csv("job_market_data.csv", index=False)
print("CSV file saved successfully.")

csv_data = pd.read_csv("job_market_data.csv")
print(csv_data.head())

cities = ["bangalore", "hyderabad", "pune", "chennai", "mumbai",
"delhi"]
all_jobs = []

for city in cities:
    city_slug = make_slug(city)
    url = f"https://www.naukri.com/{job_slug}-jobs-in-{city_slug}"
    print("Scraping:", city)
```

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
driver.get(url)

time.sleep(4)

city_cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-
tuple")
print("Jobs found:", len(city_cards))

for card in city_cards[:10]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
                        default="Not Disclosed")
    all_jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": city.title(),
        "Salary": salary
    })

city_df = pd.DataFrame(all_jobs)
city_df = city_df.drop_duplicates().reset_index(drop=True)
print("Total jobs collected:", len(city_df))

location_analysis = city_df["Location"].value_counts().reset_index()
location_analysis.columns = ["Location", "Number of Jobs"]
print(location_analysis)

plt.figure(figsize=(10, 5))
plt.bar(location_analysis["Location"], location_analysis["Number of
Jobs"])
plt.title("Data Scientist Job Demand by Location")
plt.xlabel("Location")
plt.ylabel("Number of Job Listings")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("job_demand_by_location.png", dpi=300,
bbox_inches="tight")
plt.show()

city_df.to_csv("job_market_multi_city.csv", index=False)
print("Multi-city dataset saved successfully.")

print("ONLINE JOB MARKET ANALYTICS")
print("Job Role:", JOB_ROLE)
print("Total Jobs Analyzed:", len(city_df))
print("Cities Analyzed:", len(location_analysis))
```

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

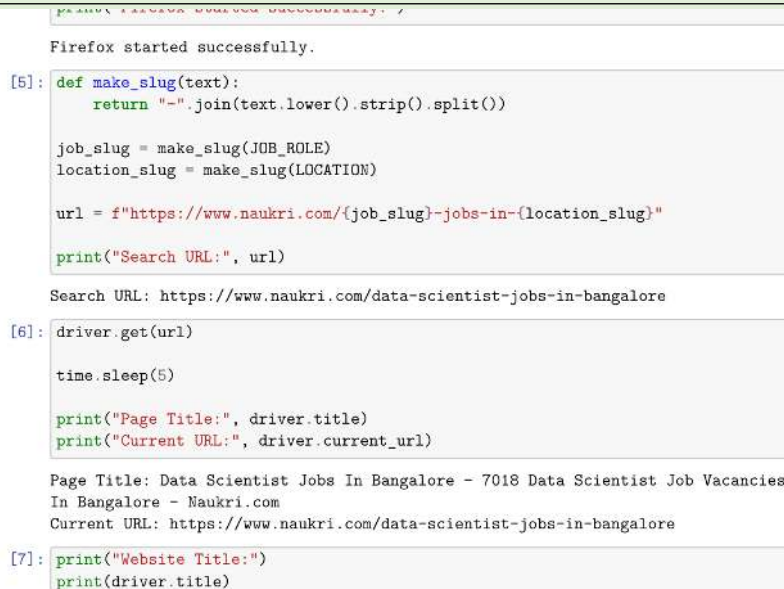
Week No.: 5

Program S. No. 10

**Output
Snapshots
With
Explanation/
Observations**

```
print(location_analysis.to_string(index=False))

driver.quit()
print("Browser closed successfully.")
```



```

# ===== OUTPUT =====
Firefox started successfully.
[5]: def make_slug(text):
      return "-".join(text.lower().strip().split())

      job_slug = make_slug(JOB_ROLE)
      location_slug = make_slug(LOCATION)

      url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"
      print("Search URL:", url)

      Search URL: https://www.naukri.com/data-scientist-jobs-in-bangalore

[6]: driver.get(url)

      time.sleep(5)

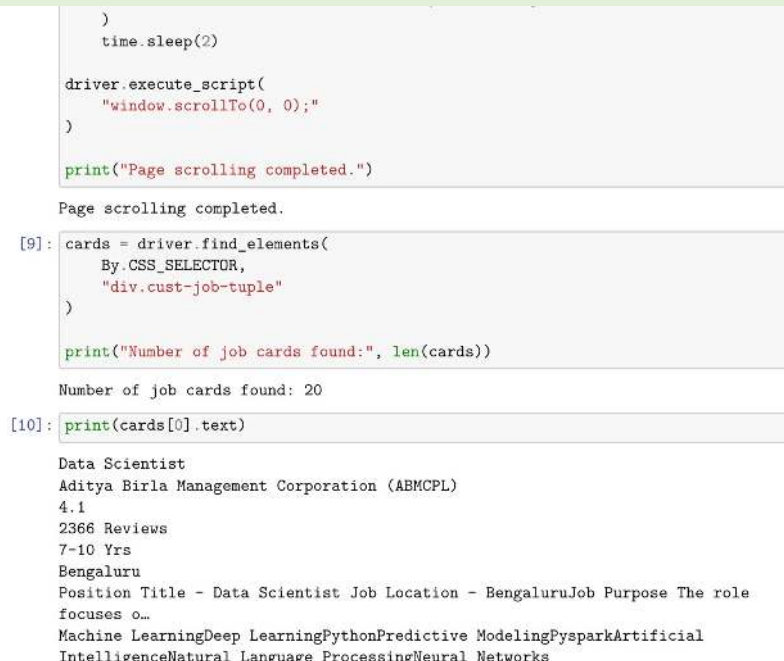
      print("Page Title:", driver.title)
      print("Current URL:", driver.current_url)

      Page Title: Data Scientist Jobs In Bangalore - 7018 Data Scientist Job Vacancies
      In Bangalore - Naukri.com
      Current URL: https://www.naukri.com/data-scientist-jobs-in-bangalore

[7]: print("Website Title:")
      print(driver.title)
  
```

Fig. 1.1: Search URL and Job Portal Validation

The Selenium session generated the Naukri search URL for Data Scientist jobs in Bangalore and successfully retrieved the page title and current URL, confirming that the dynamic job portal was opened correctly.



```

      )
      time.sleep(2)

      driver.execute_script(
          "window.scrollTo(0, 0);"
      )

      print("Page scrolling completed.")

      Page scrolling completed.

[9]: cards = driver.find_elements(
      By.CSS_SELECTOR,
      "div.cust-job-tuple"
      )

      print("Number of job cards found:", len(cards))

      Number of job cards found: 20

[10]: print(cards[0].text)

      Data Scientist
      Aditya Birla Management Corporation (ABMCPL)
      4.1
      2366 Reviews
      7-10 Yrs
      Bengaluru
      Position Title - Data Scientist Job Location - BengaluruJob Purpose The role
      focuses o...
      Machine LearningDeep LearningPythonPredictive ModelingPysparkArtificial
      IntelligenceNatural Language ProcessingNeural Networks
  
```

Fig. 1.2: Detection of Dynamic Job Cards

After scrolling the page, Selenium detected 20 job-card elements. The

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

first job card displays details such as job title, company, experience, location and associated skills.

```
jobs.append({
    "Job Title": title,
    "Company Name": company,
    "Location": location,
    "Salary": salary
})

print("Jobs scraped:", len(jobs))

Jobs scraped: 20

[14]: jobs[:3]

[14]: [{'Job Title': 'Data Scientist',
      'Company Name': 'Aditya Birla Management Corporation (ABMCPL)',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist/Software Developer',
      'Company Name': 'Xylem Water Solutions',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist III',
      'Company Name': 'Walmart',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'}]

[15]: df = pd.DataFrame(jobs)

df

[15]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist
8	Data Scientist GenAI Agentic AI LLM RAG
9	Data Scientist AIML
10	Data Scientist/Python/MLOps- 4th Sep(Fri)-6+ y...
11	Data scientist III
12	Data Scientist
13	Data Scientist/ Data Analyst
14	Walk-in Data Scientist

Fig. 1.3: Scraped Job Records and Pandas DataFrame

The scraper collected 20 listings and extracted the required Job Title, Company Name, Location and Salary fields. The records were then converted into a structured Pandas DataFrame.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
7           Pune, Bengaluru Not Disclosed
8 Hybrid - Pune, Bengaluru, Delhi / NCR Not Disclosed
9           Hyderabad, Chennai, Bengaluru Not Disclosed

[18]: print("Dataset Shape:", df.shape)

      print("\nColumn Names:")
      print(df.columns.tolist())

      print("\nTotal Records:", len(df))

Dataset Shape: (20, 4)

Column Names:
['Job Title', 'Company Name', 'Location', 'Salary']

Total Records: 20

[19]: print("Salary Information:")

      print(df["Salary"].value_counts())

Salary Information:
Salary
Not Disclosed      16
20-30 Lacs PA       1
15-20 Lacs PA       1
14-20 Lacs PA       1
0-18 Lacs PA        1
Name: count, dtype: int64

[20]: df.to_csv(
      "job_market_data.csv",
```

Fig. 1.4: Dataset Structure, Salary Availability and CSV Creation

The initial dataset contains 20 records and 4 required columns. Salary information was unavailable for most listings, while a few postings contained salary ranges. The cleaned DataFrame was saved successfully as job_market_data.csv.

```
      index=False
    )
    print("CSV file saved successfully.")

CSV file saved successfully.

[21]: csv_data = pd.read_csv(
      "job_market_data.csv"
    )

    csv_data.head()

[21]:      Job Title \
0           Data Scientist
1  Data Scientist/Software Developer
2           Data Scientist III
3           Data Scientist
4 Data Scientist | Python | GenAI | LLM | RAG

      Company Name \
0 Aditya Birla Management Corporation (ABMCPL)
1           Xylem Water Solutions
2           Walmart
3           Genpact
4           Infosys

      Location      Salary
0      Bengaluru  Not Disclosed
1      Bengaluru  Not Disclosed
2      Bengaluru  Not Disclosed
3      Bengaluru  20-30 Lacs PA
```

Fig. 1.5: Verification of the Saved CSV Dataset

The generated CSV file was read back using pandas.read_csv(), and the displayed rows confirm that the job-listing data was stored correctly in

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

structured form.

```
"Company Name": company,
"Location": city.title(),
"Salary": salary
})

Scraping: bangalore
Jobs found: 20

Scraping: hyderabad
Jobs found: 20

Scraping: pune
Jobs found: 20

Scraping: chennai
Jobs found: 20

Scraping: mumbai
Jobs found: 21

Scraping: delhi
Jobs found: 21

[25]: city_df = pd.DataFrame(all_jobs)

city_df = city_df.drop_duplicates().reset_index(drop=True)

print("Total jobs collected:", len(city_df))

city_df.head(10)

Total jobs collected: 60

[25]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist

Fig. 1.6: Multi-City Job Data Collection

The same Data Scientist search was executed for Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi. A controlled sample of 10 listings per city produced 60 records for location-wise analysis.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
4          Infosys Bangalore Not Disclosed
5          Hsbc Bangalore Not Disclosed
6          ExxonMobil Bangalore Not Disclosed
7          VOIS Bangalore Not Disclosed
8          Infosys Bangalore Not Disclosed
9          Tata Consultancy Services Bangalore Not Disclosed
```

```
[26]: location_analysis = (
      city_df["Location"]
      .value_counts()
      .reset_index()
      )

      location_analysis.columns = [
          "Location",
          "Number of Jobs"
      ]

      location_analysis
```

```
[26]: Location Number of Jobs
0 Bangalore 10
1 Hyderabad 10
2 Pune 10
3 Chennai 10
4 Mumbai 10
5 Delhi 10
```

```
[27]: plt.figure(figsize=(10, 5))
```

Fig. 1.7: Location-Wise Job Analysis

Pandas value_counts() produced the location-wise summary for the six-city job sample.

```
plt.tight_layout()
plt.show()
```

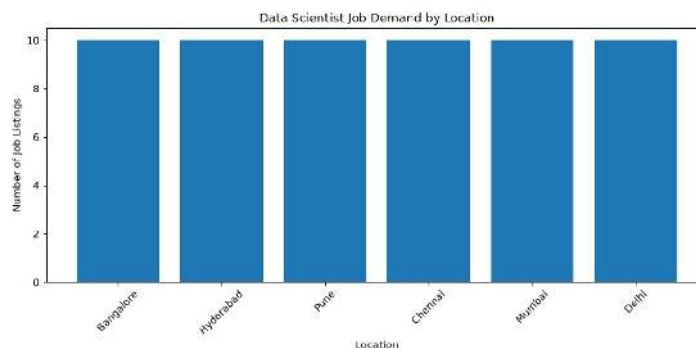


Fig. 1.8: Data Scientist Job Demand Visualization by Location

A Matplotlib bar chart visualizes the number of sampled job listings for each selected city. The graph provides a clear location-wise comparison of the collected job-market sample.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
[30]: print("ONLINE JOB MARKET ANALYTICS")
      print("-" * 35)

      print("Job Role:", JOB_ROLE)
      print("Total Jobs Analyzed:", len(city_df))
      print("Cities Analyzed:", len(location_analysis))

      print("\nLocation-wise Job Demand:")
      print(location_analysis.to_string(index=False))

      ONLINE JOB MARKET ANALYTICS
      -----
      Job Role: data scientist
      Total Jobs Analyzed: 60
      Cities Analyzed: 6

      Location-wise Job Demand:
      Location  Number of Jobs
      Bangalore          10

      Hyderabad          10
      Pune                10
      Chennai             10
      Mumbai              10
      Delhi               10

[31]: driver.quit()

      print("Browser closed successfully")
```

Fig. 1.9: Final Job Market Analytics Summary

The final output confirms that the multi-city dataset and graph were saved successfully. The analysis includes 60 sampled job records across 6 cities and displays the location-wise summary.

Conclusions

The exercise successfully demonstrated Selenium-based web scraping on a dynamic job portal. Job title, company name, location and available salary information were extracted from Naukri, organized in Pandas DataFrames, cleaned and stored as CSV files. Location-wise analysis was performed for six cities and visualized using a Matplotlib bar chart. Thus, the required job listings dataset, CSV output, location analysis and demand visualization were generated successfully.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018.
2. Python Program (GitHub link): <https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/Lab%20Codes/Week5-EP.ipynb>
3. Class lecture link: <https://drive.google.com/file/d/1HjCacfaFs49XEnuM3idduuKB0Tgjb4xI/view>

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

Title	Interactive Bar Chart with Plotly			
Problem Statement	Create an interactive bar chart showing the population of different cities.			
Objectives	• To understand the creation of interactive data visualizations using Plotly. • To organize city population data in a Pandas DataFrame. • To create an interactive bar chart for comparing population across different cities. • To sort and analyze population data to identify the highest, lowest, and average values. • To save the interactive chart as an HTML file and the dataset as a CSV file.			
Dataset URL Link / File	City		Population (Lakhs)	
	Mumbai		204	
	Delhi		167	
	Bengaluru		136	
	Hyderabad		108	
	Chennai		115	
	Kolkata		149	
	Dataset source: Manually defined in the Jupyter Notebook for this exercise; no external dataset URL is required.			
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	pandas	Creates a structured DataFrame from the city and population data.
	2	px.bar()	plotly.express	Creates an interactive bar chart from the DataFrame.
	3	update_traces()	plotly	Updates trace properties such as the position of value labels on the bars.
	4	update_layout()	plotly	Updates chart layout properties including axis titles.
	5	show()	plotly	Displays the interactive Plotly figure in the Jupyter Notebook.
	6	sort_values()	pandas	Sorts the DataFrame based on population values in ascending or descending order.
	7	idxmax()	pandas	Returns the index of the row containing

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

				the maximum population value.
	8	idxmin()	pandas	Returns the index of the row containing the minimum population value.
	9	mean()	pandas	Calculates the average population of all cities.
	10	round()	Python	Rounds the calculated average population to the required decimal places.
	11	write_html()	plotly	Saves the interactive Plotly chart as an HTML file.
	12	to_csv()	pandas	Saves the DataFrame as a CSV file without the index column.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import the Pandas library and Plotly Express for data handling and interactive visualization. 2. Create a dictionary containing the names of six Indian cities and their population values in lakhs. 3. Convert the dictionary into a Pandas DataFrame and display the dataset. 4. Check the dataset shape and print the city and population lists to verify the input data. 5. Create an interactive bar chart using Plotly Express with City on the X-axis and Population (Lakhs) on the Y-axis. 6. Display population values above the bars and provide appropriate chart and axis titles. 7. Sort the DataFrame in descending order of population using sort_values(). 8. Create and display a second interactive bar chart using the sorted population data. 9. Use idxmax() and idxmin() to identify the cities with the highest and lowest populations. 10. Calculate the average population using mean() and round the result to two decimal places. 11. Display the final population analysis summary in the notebook. 12. Save the interactive Plotly chart as city_population_chart.html and save the dataset as city_population.csv. Flow: START → Import Libraries → Create Dataset → DataFrame → Verify Data → Interactive Bar Chart → Sort Data → Sorted Chart → Population Statistics → Save HTML/CSV → STOP			
Program	import pandas as pd			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
import plotly.express as px

data = {
    "City": [
        "Mumbai",
        "Delhi",
        "Bengaluru",
        "Hyderabad",
        "Chennai",
        "Kolkata"
    ],
    "Population (Lakhs)": [
        204,
        167,
        136,
        108,
        115,
        149
    ]
}

df = pd.DataFrame(data)
df

print("Dataset Shape:", df.shape)
print("\nCities:")
print(df["City"].tolist())
print("\nPopulation:")
print(df["Population (Lakhs)"].tolist())

fig = px.bar(
    df,
    x="City",
    y="Population (Lakhs)",
    title="Population of Major Indian Cities",
    text="Population (Lakhs)"
)
fig.update_traces(textposition="outside")
fig.update_layout(
    xaxis_title="City",
    yaxis_title="Population (Lakhs)"
)
fig.show()

sorted_df = df.sort_values(
    by="Population (Lakhs)",
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
        ascending=False
    )
    sorted_df

    fig = px.bar(
        sorted_df,
        x="City",
        y="Population (Lakhs)",
        title="City-wise Population Analysis",
        text="Population (Lakhs)"
    )
    fig.update_traces(textposition="outside")
    fig.update_layout(
        xaxis_title="City",
        yaxis_title="Population (Lakhs)"
    )
    fig.show()

    highest = df.loc[
        df["Population (Lakhs)"].idxmax()
    ]
    print("City with Highest Population:")
    print(highest)

    lowest = df.loc[
        df["Population (Lakhs)"].idxmin()
    ]
    print(
        "Lowest Population:",
        lowest["City"],
        "-",
        lowest["Population (Lakhs)"],
        "Lakhs"
    )

    average_population = df["Population (Lakhs)"].mean()
    print(
        "Average Population:",
        round(average_population, 2),
        "Lakhs"
    )

    print("CITY POPULATION ANALYSIS")
    print("-" * 35)
    print("Total Cities:", len(df))
    print(
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology
(VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
"Highest Population:",
df.loc[df["Population (Lakhs)"].idxmax(), "City"]
)
print(
    "Lowest Population:",
    df.loc[df["Population (Lakhs)"].idxmin(), "City"]
)
print(
    "Average Population:",
    round(df["Population (Lakhs)"].mean(), 2),
    "Lakhs"
)

fig.write_html(
    "city_population_chart.html"
)
print("Interactive chart saved successfully.")

df.to_csv(
    "city_population.csv",
    index=False
)
print("Dataset saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following output snapshots are taken from the completed Jupyter Notebook. Each image is inserted at full content width so that the dataset, charts, code cells, and output values remain readable and are not cut off.

```
[5]:      City  Population (Lakhs)
0    Mumbai             204
1     Delhi             167
2  Bengaluru             136
3  Hyderabad             108
4    Chennai             115
5    Kolkata             149

[6]: print("Dataset Shape:", df.shape)

print("\nCities:")
print(df["City"].tolist())

print("\nPopulation:")
print(df["Population (Lakhs)"].tolist())

Dataset Shape: (6, 2)

Cities:
['Mumbai', 'Delhi', 'Bengaluru', 'Hyderabad', 'Chennai', 'Kolkata']

Population:
[204, 167, 136, 108, 115, 149]

[8]: fig = px.bar(
      df,
```

Fig. 1.1: Population Dataset and Dataset Structure

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

Explanation/Observation: The DataFrame verifies 6 cities and their corresponding population values.



Fig. 1.2: Interactive Bar Chart of Major Indian Cities

Explanation/Observation: Plotly Express generates an interactive bar chart for the original city order. The population values are displayed above the bars, with Mumbai showing the highest value of 204 lakhs in the given dataset.

```
[9]: sorted_df = df.sort_values(
    by="Population (Lakhs)",
    ascending=False
)

sorted_df
```

	City	Population (Lakhs)
0	Mumbai	204
1	Delhi	167
5	Kolkata	149
2	Bengaluru	136
4	Chennai	115
3	Hyderabad	108

Fig. 1.3: Population Data Sorted in Descending Order

Explanation/Observation: The DataFrame is sorted using `sort_values()` in descending order of population. The resulting order is Mumbai, Delhi, Kolkata, Bengaluru, Chennai, and Hyderabad.



Fig. 1.4: City-wise Population Analysis after Sorting

Explanation/Observation: The sorted bar chart displays the cities from

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

highest to lowest population, making comparison and ranking easier.

```
[11]: highest = df.loc[
      df["Population (Lakhs)"].idxmax()
    ]

    print("City with Highest Population:")
    print(highest)
```

```
City with Highest Population:
City                Mumbai
Population (Lakhs)    204
Name: 0, dtype: object
```

```
[12]: lowest = df.loc[
      df["Population (Lakhs)"].idxmin()
    ]

    print(
        "Lowest Population:",
        lowest["City"],
        "-",
        lowest["Population (Lakhs)"],
        "Lakhs"
    )
```

```
Lowest Population: Hyderabad - 108 Lakhs
```

```
[13]: average_population = df["Population (Lakhs)"].mean()

    print(
        "Average Population:",
        round(average_population, 2),
        "Lakhs"
    )
```

```
Average Population: 146.5 Lakhs
```

Fig. 1.5: Highest, Lowest, and Average Population Analysis

Explanation/Observation: The program identifies Mumbai as the city with the highest population (204 lakhs), Hyderabad as the city with the lowest population (108 lakhs), and calculates the average population as 146.5 lakhs.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
[14]: print("CITY POPULATION ANALYSIS")
      print("-" * 35)

      print("Total Cities:", len(df))

      print(
          "Highest Population:",
          df.loc[df["Population (Lakhs)"].idxmax(), "City"]
      )

      print(
          "Lowest Population:",
          df.loc[df["Population (Lakhs)"].idxmin(), "City"]
      )

      print(
          "Average Population:",
          round(df["Population (Lakhs)"].mean(), 2),
          "Lakhs"
      )

CITY POPULATION ANALYSIS
-----
Total Cities: 6
Highest Population: Mumbai
Lowest Population: Hyderabad
Average Population: 146.5 Lakhs

[15]: fig.write_html(
      "city_population_chart.html"
      )

      print("Interactive chart saved successfully.")

Interactive chart saved successfully.

[16]: df.to_csv(
      "city_population.csv",
      index=False
      )

      print("Dataset saved successfully.")

Dataset saved successfully.

[ ]:
```

Fig. 1.6: Final Analysis Summary and File Saving Output

Explanation/Observation: The final summary confirms 6 cities were analyzed, with Mumbai highest and Hyderabad lowest. The interactive chart and CSV dataset are saved successfully.

Conclusions

The experiment was successfully completed using Pandas and Plotly. The population data was organized in a DataFrame, visualized with interactive bar charts, and sorted for comparison. Mumbai had the highest population, Hyderabad the lowest, and the average population was 146.5 lakhs. The interactive chart was saved as HTML and the dataset as CSV.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science, Addison-Wesley, 2018.
2. Python Program (GitHub link): <https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/Week%206/Week-6-PP.ipynb>
3. Class lecture link: [Open Lecture File](#)

Student Reg. No: 26MML0031 Name: Hasnain Makada

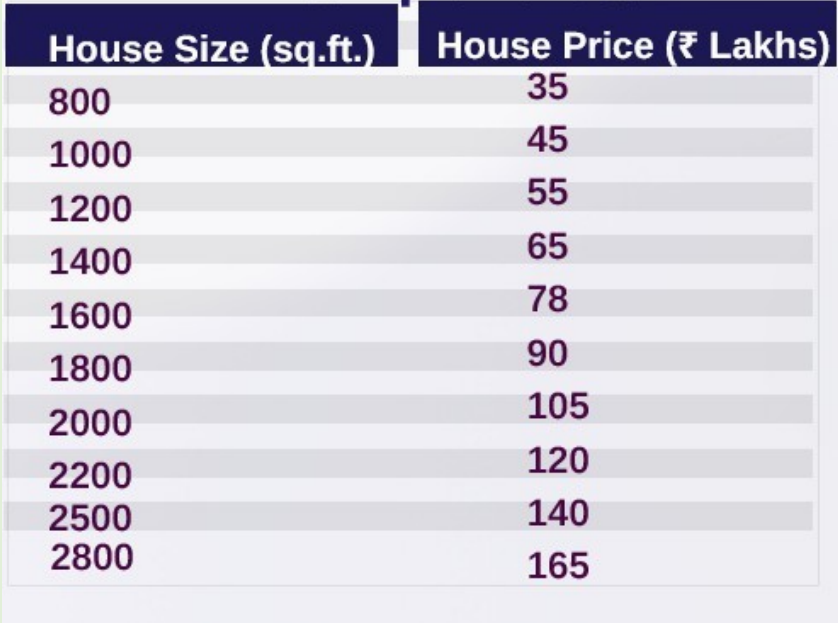
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

Title	Interactive Scatter Plot with Plotly			
Problem Statement	Create an interactive scatter plot showing the relationship between house size and price.			
Objectives	• To understand interactive scatter plot visualization using Plotly. • To organize house size and house price data using a Pandas DataFrame. • To visualize and interpret the relationship between house size and house price. • To compute descriptive measures such as minimum, maximum, average, and correlation. • To create a trendline for the relationship and save the interactive chart and dataset for further use.			
Dataset URL Link / File	 Dataset source: Manually defined in the Jupyter Notebook using the house-size and house-price values shown above; no external dataset URL is required.			
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	pandas	Creates a structured DataFrame from the house size and price data.
	2	px.scatter()	plotly.express	Creates an interactive scatter plot from the DataFrame.
	3	update_traces()	plotly	Updates marker size and text-position properties of the plotted points.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	4	update_layout()	plotly	Updates the chart layout and axis titles.
	5	show()	plotly	Displays the interactive Plotly chart inside the Jupyter Notebook.
	6	sort_values()	pandas	Sorts the dataset based on house size in ascending order.
	7	idxmax()	pandas	Returns the index of the record with the maximum house price.
	8	idxmin()	pandas	Returns the index of the record with the minimum house price.
	9	mean()	pandas	Calculates the average house price and average house size.
	10	corr()	pandas	Calculates the correlation between house size and house price.
	11	write_html()	plotly	Saves the interactive Plotly scatter plot as an HTML file.
	12	to_csv()	pandas	Saves the house-price DataFrame as a CSV file without the index column.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model		1. Import Pandas and Plotly Express for tabular data handling and interactive visualization. 2. Create a dictionary containing house sizes in square feet and corresponding house prices in lakhs. 3. Convert the dictionary into a Pandas DataFrame and display all ten records. 4. Verify the dataset shape and print the complete house-size and house-price lists. 5. Create a basic interactive scatter plot with house size on the X-axis and house price on the Y-axis. 6. Create an improved scatter plot with larger markers, value labels, title, and proper axis labels. 7. Sort the DataFrame by house size in ascending order and display the sorted dataset. 8. Use idxmax() and idxmin() to identify the most expensive and least expensive houses. 9. Calculate the average house price and average house size using mean(). 10. Calculate the correlation between house size and price using corr() to measure the strength of their relationship. 11. Create a scatter plot with an OLS trendline to visually represent the positive relationship between size and price.		

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	12. Display the final house-price analysis and save the interactive chart as HTML and the dataset as CSV. Flow: START → Import Libraries → Create Dataset → DataFrame → Verify Data → Scatter Plot → Enhanced Plot → Sort Data → Price Statistics → Average Values → Correlation → Trendline → Save HTML/CSV → STOP
Program	<pre>import pandas as pd import plotly.express as px data = { "House Size (sq.ft)": [800, 1000, 1200, 1400, 1600, 1800, 2000, 2200, 2500, 2800], "House Price (Lakhs)": [35, 45, 55, 65, 78, 90, 105, 120, 140, 165] } df = pd.DataFrame(data) df print("Dataset Shape:", df.shape) print("\nHouse Sizes:") print(df["House Size (sq.ft.)"].tolist()) print("\nHouse Prices:") print(df["House Price (Lakhs)"].tolist())</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs House Price"
)
fig.show()

fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="Relationship Between House Size and Price",
    text="House Price (Lakhs)"
)
fig.update_traces(
    marker=dict(size=12),
    textposition="top center"
)
fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price (Lakhs)"
)
fig.show()

sorted_df = df.sort_values(
    by="House Size (sq.ft.)",
    ascending=True
)
sorted_df

highest = df.loc[
    df["House Price (Lakhs)"].idxmax()
]
print("Most Expensive House:")
print(highest)

lowest = df.loc[
    df["House Price (Lakhs)"].idxmin()
]
print(
    "Least Expensive House:",
    lowest["House Size (sq.ft.)"],
    "sq.ft. - ",
    lowest["House Price (Lakhs)"],
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
"Lakhs"
)

average_price = df["House Price (Lakhs)"].mean()
print(
    "Average House Price:",
    round(average_price, 2),
    "Lakhs"
)

average_size = df["House Size (sq.ft.)"].mean()
print(
    "Average House Size:",
    round(average_size, 2),
    "sq.ft."
)

correlation = df[
    "House Size (sq.ft.)"
].corr(
    df["House Price (Lakhs)"]
)
print(
    "Correlation between House Size and Price:",
    round(correlation, 3)
)

fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs Price with Trendline",
    trendline="ols"
)
fig.update_traces(
    marker=dict(size=12)
)
fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price (Lakhs)"
)
fig.show()

print("HOUSE PRICE ANALYSIS")
print("-" * 40)
print("Total Houses:", len(df))
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology
(VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	<pre> print("Smallest House:", df["House Size (sq.ft.)"].min(), "sq.ft.") print("Largest House:", df["House Size (sq.ft.)"].max(), "sq.ft.") print("Lowest Price: ", df["House Price (Lakhs)"].min(), "Lakhs") print("Highest Price: ", df["House Price (Lakhs)"].max(), "Lakhs") print("Average Price: ", round(df["House Price (Lakhs)"].mean(), 2), "Lakhs") print("Correlation:", round(correlation, 3)) fig.write_html("house_size_price_scatter.html") print("Interactive scatter plot saved successfully.") df.to_csv("house_price_data.csv", index=False) print("Dataset saved successfully.") </pre>
Output Snapshots With Explanation/ Observations	The following snapshots are taken from the completed Jupyter Notebook. The images retain the complete visible code/output blocks from each source page; only unused white margins were removed so no program output is cut off.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
165
    ]
}

[3]: df = pd.DataFrame(data)

df

[3]:   House Size (sq.ft.)  House Price (Lakhs)
0                800             35
1               1000             45
2               1200             55
3               1400             65
4               1600             78
5               1800             90
6               2000            105
7               2200            120
8               2500            140
9               2800            165

[4]: print("Dataset Shape:", df.shape)

print("\nHouse Sizes:")
print(df["House Size (sq.ft.)"].tolist())

print("\nHouse Prices:")
print(df["House Price (Lakhs)"].tolist())

Dataset Shape: (10, 2)

House Sizes:
[800, 1000, 1200, 1400, 1600, 1800, 2000, 2200, 2500, 2800]

House Prices:
[35, 45, 55, 65, 78, 90, 105, 120, 140, 165]

[5]: fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs House Price"
)

fig.show()
```

2

Fig. 1.1: DataFrame Creation, Dataset Verification, and Basic Scatter Plot Code

Explanation/Observation: The dataset is converted into a Pandas DataFrame with 10 rows and 2 columns. The complete house-size and house-price lists are verified, and the code for the initial interactive scatter plot is executed.

Student Reg. No: 26MML0031 Name: Hasnain Makada

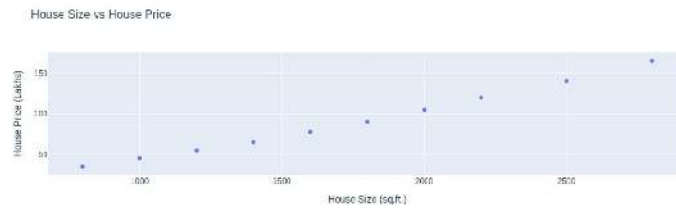
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12



```
[6]: fig = px.scatter(  
    df,  
    x="House Size (sq.ft.)",  
    y="House Price (Lakhs)",  
    title="Relationship Between House Size and Price",  
    text="House Price (Lakhs)"  
)  
  
fig.update_traces(  
    marker=dict(size=12),  
    textposition="top center"  
)  
  
fig.update_layout(  
    xaxis_title="House Size (sq.ft.)",  
    yaxis_title="House Price ( Lakhs)"  
)  
  
fig.show()
```



3

Fig. 1.2: Interactive and Enhanced Scatter Plots

Explanation/Observation: The first scatter plot shows house size versus price. The enhanced Plotly chart adds larger markers and price labels, making the strong upward relationship between size and price visually clear.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
[7]: sorted_df = df.sort_values(
      by="House Size (sq.ft.)",
      ascending=True
    )

sorted_df

[7]:   House Size (sq.ft.)  House Price (Lakhs)
0              800             35
1             1000             45
2             1200             55
3             1400             65
4             1600             78
5             1800             90
6             2000            105
7             2200            120
8             2500            140
9             2800            165

[8]: highest = df.loc[
      df["House Price (Lakhs)"].idxmax()
    ]

print("Most Expensive House:")
print(highest)

Most Expensive House:
House Size (sq.ft.)    2800
House Price (Lakhs)     165
Name: 9, dtype: int64

[9]: lowest = df.loc[
      df["House Price (Lakhs)"].idxmin()
    ]

print(
    "Least Expensive House:",
    lowest["House Size (sq.ft.)"],
    "sq.ft. - ",
    lowest["House Price (Lakhs)"],
    "Lakhs"
)

Least Expensive House: 800 sq.ft. - 35 Lakhs

[10]: average_price = df["House Price (Lakhs)"].mean()

print()
```

4

Fig. 1.3: Sorted Dataset and Extreme Price Analysis

Explanation/Observation: The dataset is sorted by house size in ascending order. The program identifies the 2800 sq.ft. house priced at 165 lakhs as the most expensive and the 800 sq.ft. house priced at 35 lakhs as the least expensive.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
"Average House Price:",
round(average_price, 2),
"Lakhs"
)

Average House Price: 89.8 Lakhs

[11]: average_size = df["House Size (sq.ft.)"].mean()

print(
    "Average House Size:",
    round(average_size, 2),
    "sq.ft."
)

Average House Size: 1730.0 sq.ft.

[12]: correlation = df[
    "House Size (sq.ft.)"
].corr(
    df["House Price (Lakhs)"]
)

print(
    "Correlation between House Size and Price:",
    round(correlation, 3)
)

Correlation between House Size and Price: 0.996

[14]: fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs Price with Trendline",
    trendline="ols"
)

fig.update_traces(
    marker=dict(size=12)
)

fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price ( Lakhs)"
)

fig.show()
```

5

Fig. 1.4: Average Values, Correlation, and Trendline Program

Explanation/Observation: The average house price is 89.8 lakhs and the average house size is 1730 sq.ft. The calculated correlation is 0.996, indicating a very strong positive relationship. The trendline scatter-plot code is also shown.

Student Reg. No: 26MML0031 Name: Hasnain Makada

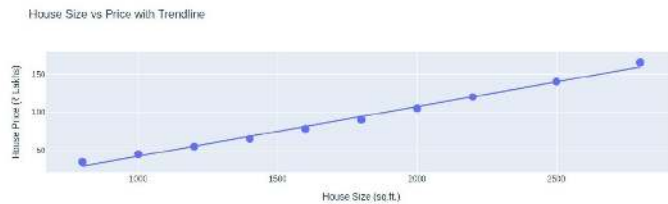
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12



```
[15]: print("HOUSE PRICE ANALYSIS")
      print("-" * 40)

      print("Total Houses:", len(df))

      print(
        "Smallest House:",
        df["House Size (sq.ft.)"].min(),
        "sq.ft."
      )

      print(
        "Largest House:",
        df["House Size (sq.ft.)"].max(),
        "sq.ft."
      )

      print(
        "Lowest Price: ",
        df["House Price (Lakhs)"].min(),
        "Lakhs"
      )

      print(
        "Highest Price: ",
        df["House Price (Lakhs)"].max(),
        "Lakhs"
      )

      print(
        "Average Price: ",
        round(df["House Price (Lakhs)"].mean(), 2),
        "Lakhs"
      )
```

6

Fig. 1.5: Scatter Plot with OLS Trendline and Final Analysis Code

Explanation/Observation: The Plotly scatter plot with an OLS trendline confirms that house price rises strongly with house size. The final analysis code computes the smallest/largest houses and lowest/highest/average prices.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```

)

print(
    "Correlation:",
    round(correlation, 3)
)

HOUSE PRICE ANALYSIS
-----

Total Houses: 10
Smallest House: 800 sq.ft.
Largest House: 2800 sq.ft.
Lowest Price: 35 Lakhs
Highest Price: 165 Lakhs
Average Price: 89.8 Lakhs
Correlation: 0.996

[16]: fig.write_html(
        "house_size_price_scatter.html"
    )

    print("Interactive scatter plot saved successfully.")

Interactive scatter plot saved successfully.

[17]: df.to_csv(
        "house_price_data.csv",
        index=False
    )

    print("Dataset saved successfully.")

Dataset saved successfully.

[ ]:

```

7

Fig. 1.6: Final House Price Analysis and File Saving Output

Explanation/Observation: The final output confirms 10 houses, sizes from 800 to 2800 sq.ft., prices from 35 to 165 lakhs, an average price of 89.8 lakhs, and correlation of 0.996. The interactive HTML chart and CSV dataset are saved successfully.

Conclusions

The experiment was successfully completed using Pandas and Plotly. House size and price data were organized in a Pandas DataFrame and visualized through interactive scatter plots. The analysis found an average house size of 1730 sq.ft., an average price of 89.8 lakhs, and a correlation of 0.996, showing a very strong positive relationship between house size and price. An OLS trendline further confirmed this relationship. The interactive

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	chart and dataset were successfully saved as HTML and CSV files.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018. 2. Python Program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%206/Week-6-PP2.ipynb 3. Class lecture link: Open Lecture File

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

Title	Interactive Scatter Plot with Bokeh			
Problem Statement	Create an interactive scatter plot showing the relationship between petal length and petal width from the Iris dataset using Bokeh.			
Objectives	• To understand interactive scatter plot visualization using Bokeh. • To load and organize the Iris dataset using Pandas and scikit-learn. • To visualize the relationship between petal length and petal width. • To differentiate Iris species using separate colors and an interactive legend. • To use Bokeh hover, zoom, pan, reset and save tools for interactive analysis.			
Dataset URL Link / File	Dataset: Built-in Iris dataset loaded using sklearn.datasets.load_iris(). Selected attributes: Petal Length (cm), Petal Width (cm), and Species.			
	Species		Samples	
	setosa		50	
	versicolor		50	
	virginica		50	
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads the built-in Iris dataset.
	2	rename()	Pandas	Renames dataset columns for clearer labels.
	3	map()	Pandas	Maps numeric target values to species names.
	4	DataFrame()	Pandas	Represents the selected Iris attributes in tabular form.
	5	ColumnData Source()	Bokeh	Creates a Bokeh-friendly data source.
	6	figure()	Bokeh	Creates the interactive plotting canvas.
	7	scatter()	Bokeh	Plots petal length against petal width.
	8	HoverTool()	Bokeh	Displays species and measurements on hover.
	9	add_tools()	Bokeh	Adds the hover tool to the Bokeh figure.
	10	value_counts()	Pandas	Counts the records belonging to each Iris species.
	11	corr()	Pandas	Calculates correlation between petal length and width.
	12	output_file()/save()	Bokeh	Saves the interactive visualization as an HTML file.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, the Iris dataset utility, and required Bokeh modules. 2. Load the Iris dataset and rename the petal measurement columns for clarity. 3. Convert numeric target values into the species names setosa, versicolor and virginica. 4. Create a DataFrame containing Petal Length, Petal Width and Species. 5. Display sample records, dataset shape, species counts and correlation for verification. 6. Create a Bokeh figure with interactive pan, zoom, reset and save tools. 7. Plot one scatter series for each Iris species using a ColumnDataSource. 8. Add HoverTool so that species and measurements appear when the pointer is placed over a point. 9. Enable an interactive legend so species can be hidden or displayed. 10. Save the interactive Bokeh chart as an HTML file and verify successful execution. Flow: START -> Load Iris Dataset -> Prepare DataFrame -> ColumnDataSource -> Bokeh Figure -> Scatter Plot -> Hover/Legend -> Save HTML -> STOP
Program	<pre>import pandas as pd from sklearn.datasets import load_iris from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool from bokeh.palettes import Category10 # Load the Iris dataset iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)" }) df["Species"] = df["target"].map(dict(enumerate(iris.target_names))) df = df[["Petal Length (cm)", "Petal Width (cm)", "Species"]] print(df.head(10)) print("Dataset Shape:", df.shape) print("Species Counts:") print(df["Species"].value_counts()) correlation = df["Petal Length (cm)"].corr(df["Petal Width (cm)"]) print("Correlation:", round(correlation, 3)) # Create the interactive Bokeh figure p = figure(title="Interactive Scatter Plot: Iris Petal Length vs Petal Width", x_axis_label="Petal Length (cm)", y_axis_label="Petal Width (cm)",</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

```
width=900,
height=520,
tools="pan,wheel_zoom,box_zoom,reset,save"
)

colors = Category10[3]
for color, species in zip(colors, iris.target_names):
    subset = df[df["Species"] == species]
    source = ColumnDataSource(subset)
    p.scatter(
        x="Petal Length (cm)",
        y="Petal Width (cm)",
        source=source,
        size=9,
        color=color,
        alpha=0.75,
        legend_label=species
    )

hover = HoverTool(tooltips=[
    ("Species", "@Species"),
    ("Petal Length", "@{Petal Length (cm)}{0.0} cm"),
    ("Petal Width", "@{Petal Width (cm)}{0.0} cm")
])
p.add_tools(hover)
p.legend.title = "Species"
p.legend.location = "top_left"
p.legend.click_policy = "hide"

output_file("program13_iris_scatter.html")
save(p)
print("Interactive scatter plot saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following outputs were generated by executing the complete Program 13 code. The static chart image represents the interactive Bokeh visualization; the HTML file retains hover, zoom, pan and reset functionality.

Jupyter Output - Iris Dataset Summary

```
First 10 rows: Petal Length (cm), Petal Width (cm), Species
1.4  0.2  setosa
1.4  0.2  setosa
1.3  0.2  setosa
1.5  0.2  setosa
1.4  0.2  setosa
...
Dataset Shape: (150, 3)
Species Counts: setosa 50 | versicolor 50 | virginica 50
Correlation (Petal Length vs Petal Width): 0.963
```

Fig. 1.1: Iris Dataset and Correlation Summary

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

The output verifies that 150 Iris samples were loaded, with 50 records for each species. The petal length-petal width correlation is approximately 0.963, indicating a strong positive relationship.

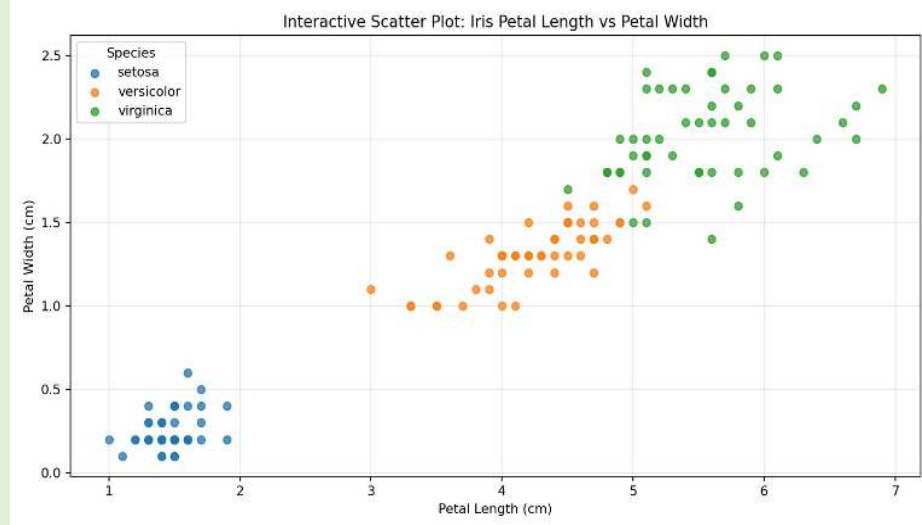


Fig. 1.2: Interactive Scatter Plot Output

The scatter plot shows three clearly distinguishable Iris species. Setosa forms a compact cluster at lower petal measurements, while versicolor and virginica occupy progressively larger petal-length and petal-width ranges.

Jupyter Output - Program 13 Final Status

```
Interactive scatter plot created successfully using Bokeh.
HTML file saved: program13_iris_scatter.html
Total records visualized: 150
Species displayed: setosa, versicolor, virginica
Hover, zoom, pan, reset and save tools are enabled.
```

Fig. 1.3: Program 13 Execution and HTML Save Confirmation

The final execution confirms successful generation of the interactive Bokeh scatter plot and its HTML output. Hover, zoom, pan, reset and save tools are available in the saved interactive file.

Conclusions

The experiment was completed successfully using Pandas, scikit-learn and Bokeh. The Iris dataset was loaded and organized into a DataFrame containing petal length, petal width and species. The interactive scatter plot clearly demonstrated a strong positive relationship between petal length and petal width and visually separated the three Iris species. Bokeh interaction tools further enabled hovering, zooming, panning and legend-based filtering, making the visualization suitable for exploratory analysis.

References

1. Bokeh Documentation: [Open Link](#)
2. Pandas Documentation: [Open Link](#)
3. scikit-learn Iris Dataset Documentation: [Open Link](#)
4. Python Program (GitHub link): [Open Week 7 Notebook](#)
5. Class lecture link: [Open Lecture Link](#)

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

Title	Interactive Line Plot with Bokeh			
Problem Statement	Create an interactive line plot showing the daily temperatures over a week using Bokeh.			
Objectives	• To understand interactive line visualization using Bokeh. • To organize daily temperature values in a Pandas DataFrame. • To visualize day-wise temperature changes over one week. • To identify the highest, lowest and average weekly temperature. • To use hover, zoom, pan, reset and save tools for interactive time-series analysis.			
Dataset URL Link / File	Dataset: Manually defined weekly temperature readings for seven consecutive days.			
	Day		Temperature (°C)	
	Monday		29	
	Tuesday		31	
	Wednesday		30	
	Thursday		32	
	Friday		34	
	Saturday		33	
	Sunday		31	
Functions Details	Sl. No.	Function	Package	Description of function
	1	DataFrame()	Pandas	Creates a tabular dataset from the day and temperature values.
	2	ColumnData Source()	Bokeh	Converts the DataFrame into a Bokeh data source.
	3	figure()	Bokeh	Creates the interactive Bokeh plotting canvas.
	4	line()	Bokeh	Draws the line connecting daily temperatures.
	5	scatter()	Bokeh	Adds visible markers for each day.
	6	HoverTool()	Bokeh	Displays day and temperature values on hover.
	7	add_tools()	Bokeh	Adds the hover tool to the figure.
	8	idxmax()	Pandas	Locates the day with the maximum temperature.
	9	idxmin()	Pandas	Locates the day with the minimum temperature.
	10	mean()	Pandas	Calculates the average weekly temperature.
	11	output_file()	Bokeh	Defines the HTML output filename.
	12	save()	Bokeh	Saves the interactive line plot as an HTML file.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas and the required Bokeh modules. 2. Define the seven days of the week and their corresponding temperature values. 3. Create a Pandas DataFrame and display it for verification. 4. Calculate the highest, lowest and average temperature values. 5. Convert the DataFrame into a Bokeh ColumnDataSource. 6. Create a Bokeh figure using the days as categorical X-axis values. 7. Plot the temperature trend using a line and add markers for each day. 8. Add HoverTool to display the exact day and temperature interactively. 9. Save the completed visualization as an HTML file and verify successful execution. Flow: START -> Create Weekly Dataset -> DataFrame -> Temperature Statistics -> ColumnDataSource -> Bokeh Figure -> Line + Markers -> Hover -> Save HTML -> STOP
Program	<pre>import pandas as pd from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool # Weekly temperature dataset days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"] temperatures = [29, 31, 30, 32, 34, 33, 31] df = pd.DataFrame({"Day": days, "Temperature (°C)": temperatures}) print(df) highest_row = df.loc[df["Temperature (°C)"].idxmax()] lowest_row = df.loc[df["Temperature (°C)"].idxmin()] average_temp = df["Temperature (°C)"].mean() print("Highest Temperature:", highest_row["Day"], "-", highest_row["Temperature (°C)"], "°C") print("Lowest Temperature:", lowest_row["Day"], "-", lowest_row["Temperature (°C)"], "°C") print("Average Temperature:", round(average_temp, 2), "°C") source = ColumnDataSource(df) p = figure(x_range=days, title="Interactive Line Plot: Daily Temperature Over a Week", x_axis_label="Day", y_axis_label="Temperature (°C)", width=900, height=520, tools="pan,wheel_zoom,box_zoom,reset,save")</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

```
p.line(x="Day", y="Temperature (°C)", source=source, line_width=3)
p.scatter(x="Day", y="Temperature (°C)", source=source, size=10)
```

```
hover = HoverTool(tooltips=[
    ("Day", "@Day"),
    ("Temperature", "@{Temperature (°C)} °C")
])
p.add_tools(hover)
p.xgrid.grid_line_color = None
```

```
output_file("program14_temperature_line.html")
save(p)
print("Interactive line plot saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following outputs were generated from the completed Program 14. The chart image is a static view of the interactive Bokeh line plot, while the saved HTML output contains the full interactive tools.

Jupyter Output - Weekly Temperature Summary

```
Weekly Temperature Dataset:
Monday 29 °C | Tuesday 31 °C | Wednesday 30 °C
Thursday 32 °C | Friday 34 °C | Saturday 33 °C | Sunday 31 °C
Highest Temperature: Friday - 34 °C
Lowest Temperature: Monday - 29 °C
Average Temperature: 31.43 °C
```

Fig. 2.1: Weekly Temperature Dataset and Statistical Summary

The dataset contains seven daily readings. Friday records the highest temperature of 34 °C, Monday records the lowest temperature of 29 °C, and the weekly average is approximately 31.43 °C.

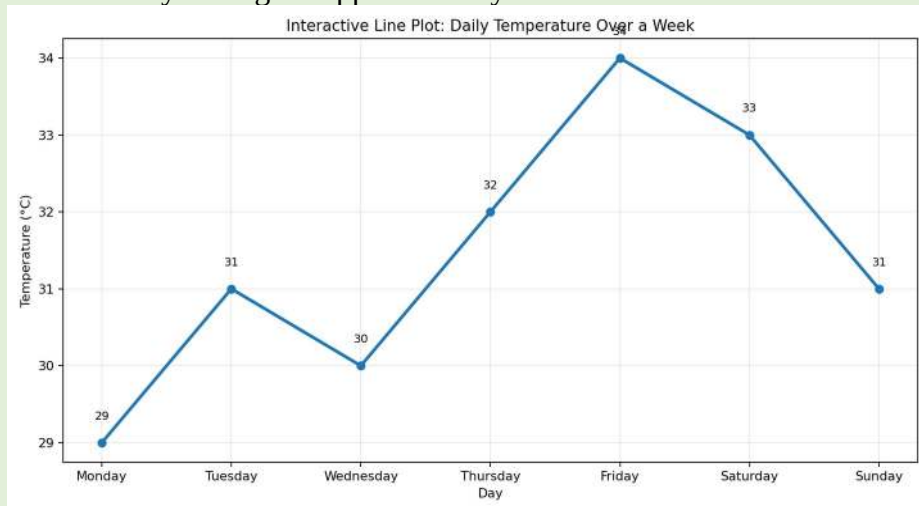


Fig. 2.2: Interactive Line Plot Output

The line plot shows the day-wise temperature variation across the week. The temperature rises overall from Monday to Friday, reaches its maximum on Friday, and decreases during the weekend.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

	Jupyter Output - Program 14 Final Status Interactive line plot created successfully using Bokeh. HTML file saved: program14_temperature_line.html Days visualized: 7 Hover, zoom, pan, reset and save tools are enabled. Fig. 2.3: Program 14 Execution and HTML Save Confirmation The execution confirms successful creation and saving of the Bokeh line plot. The resulting HTML file supports hover, zoom, pan, reset and save interactions.
Conclusions	The exercise was completed successfully using Pandas and Bokeh. Daily temperature values for one week were organized in a DataFrame and visualized using an interactive line plot. Statistical analysis identified Friday as the hottest day, Monday as the coolest day, and the average weekly temperature as approximately 31.43 °C. The interactive Bokeh output makes the weekly trend easy to inspect using hover and navigation tools.
References	1. Bokeh Documentation: Open Link 2. Pandas Documentation: Open Link 3. Python Program (GitHub link): Open Week 7 Notebook 4. Class lecture link: Open Lecture Link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

Title	Interactive Bar Chart with Bokeh			
Problem Statement	Create an interactive bar chart showing the sales figures of different products using Bokeh.			
Objectives	• To understand interactive categorical bar-chart visualization using Bokeh. • To organize product-wise sales values in a Pandas DataFrame. • To compare sales performance across different products. • To calculate total, highest and average product sales. • To use Bokeh hover, zoom, pan, reset and save tools for interactive analysis.			
Dataset URL Link / File	Dataset: Manually defined product-wise sales values for six products.			
	Product		Sales (Units)	
	Laptop		125	
	Smartphone		210	
	Tablet		95	
	Headphones		170	
	Smartwatch		140	
	Camera		80	
Functions Details	Sl. No.	Function	Package	Description of function
	1	DataFrame()	Pandas	Creates a structured product-sales DataFrame.
	2	ColumnData Source()	Bokeh	Converts the DataFrame into a Bokeh data source.
	3	figure()	Bokeh	Creates the categorical plotting canvas.
	4	vbar()	Bokeh	Draws one vertical bar for each product.
	5	HoverTool()	Bokeh	Displays product and sales values on hover.
	6	add_tools()	Bokeh	Adds the hover tool to the figure.
	7	idxmax()	Pandas	Locates the product with the highest sales.
	8	max()	Pandas	Returns the highest sales value.
	9	sum()	Pandas	Calculates the total number of units sold.
	10	mean()	Pandas	Calculates the average sales per product.
	11	output_file()	Bokeh	Defines the interactive HTML output filename.
	12	save()	Bokeh	Saves the completed Bokeh bar chart as HTML.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas and the required Bokeh modules. 2. Define the product names and their corresponding sales figures. 3. Create and display a Pandas DataFrame for verification. 4. Calculate total sales, the highest-selling product and average sales. 5. Convert the DataFrame to a Bokeh ColumnDataSource. 6. Create a categorical Bokeh figure with product names on the X-axis. 7. Use vbar() to display the product-wise sales values. 8. Add HoverTool so exact sales values are visible interactively. 9. Save the completed interactive bar chart as an HTML file and verify successful execution. Flow: START -> Create Product Dataset -> DataFrame -> Sales Statistics -> ColumnDataSource -> Bokeh Figure -> Vertical Bars -> Hover -> Save HTML -> STOP
Program	<pre>import pandas as pd from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool # Product sales dataset products = ["Laptop", "Smartphone", "Tablet", "Headphones", "Smartwatch", "Camera"] sales = [125, 210, 95, 170, 140, 80] df = pd.DataFrame({"Product": products, "Sales (Units)": sales}) print(df) highest_row = df.loc[df["Sales (Units)"].idxmax()] total_sales = df["Sales (Units)"].sum() average_sales = df["Sales (Units)"].mean() print("Total Sales:", total_sales, "units") print("Highest Sales:", highest_row["Product"], "-", highest_row["Sales (Units)"], "units") print("Average Sales:", round(average_sales, 2), "units") source = ColumnDataSource(df) p = figure(x_range=products, title="Interactive Bar Chart: Product Sales", x_axis_label="Product", y_axis_label="Sales (Units)", width=900, height=520, tools="pan,wheel_zoom,box_zoom,reset,save") p.vbar(x="Product", top="Sales (Units)", source=source, width=0.65)</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

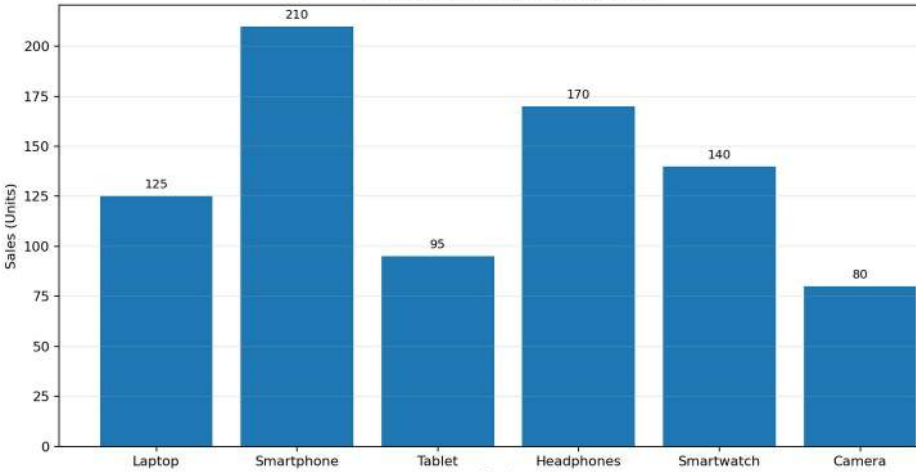
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

	<pre> hover = HoverTool(tooltips=[("Product", "@Product"), ("Sales", "@{Sales (Units)} units")]) p.add_tools(hover) p.xgrid.grid_line_color = None output_file("program15_product_sales_bar.html") save(p) print("Interactive bar chart saved successfully.") </pre>														
Output Snapshots With Explanation/ Observations	The following outputs were generated by executing Program 15. The static chart image presents the product-wise sales comparison, while the generated Bokeh HTML file preserves the interactive functionality. Jupyter Output - Product Sales Summary Product Sales Dataset: Laptop 125 Smartphone 210 Tablet 95 Headphones 170 Smartwatch 140 Camera 80 Total Sales: 820 units Highest Sales: Smartphone - 210 units Average Sales: 136.67 units Fig. 3.1: Product Sales Dataset and Statistical Summary The product dataset contains six sales values. Total sales are 820 units, Smartphone has the highest sales at 210 units, and average sales are approximately 136.67 units per product. Interactive Bar Chart: Product Sales  <table border="1" style="margin-top: 10px;"> <thead> <tr> <th>Product</th> <th>Sales (Units)</th> </tr> </thead> <tbody> <tr> <td>Laptop</td> <td>125</td> </tr> <tr> <td>Smartphone</td> <td>210</td> </tr> <tr> <td>Tablet</td> <td>95</td> </tr> <tr> <td>Headphones</td> <td>170</td> </tr> <tr> <td>Smartwatch</td> <td>140</td> </tr> <tr> <td>Camera</td> <td>80</td> </tr> </tbody> </table> Fig. 3.2: Interactive Bar Chart Output The bar chart compares sales across all six products. Smartphone is the highest-selling product, followed by Headphones, while Camera records the lowest sales.	Product	Sales (Units)	Laptop	125	Smartphone	210	Tablet	95	Headphones	170	Smartwatch	140	Camera	80
Product	Sales (Units)														
Laptop	125														
Smartphone	210														
Tablet	95														
Headphones	170														
Smartwatch	140														
Camera	80														

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

	Jupyter Output - Program 15 Final Status Interactive bar chart created successfully using Bokeh. HTML file saved: program15_product_sales_bar.html Products visualized: 6 Hover, zoom, pan, reset and save tools are enabled. <i>Fig. 3.3: Program 15 Execution and HTML Save Confirmation</i> The final execution confirms successful generation and saving of the Bokeh product-sales bar chart. The HTML output supports hover and navigation tools for interactive inspection.
Conclusions	The exercise was completed successfully using Pandas and Bokeh. Product sales data were organized in a DataFrame and visualized through an interactive vertical bar chart. Smartphone recorded the highest sales with 210 units, total sales were 820 units, and average sales were approximately 136.67 units. The visualization provides a clear product-wise comparison and the interactive Bokeh tools improve data exploration.
References	1. Bokeh Documentation: Open Link 2. Pandas Documentation: Open Link 3. Python Program (GitHub link): Open Week 7 Notebook 4. Class lecture link: Open Lecture Link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly.			
Objectives	• Load and verify the Iris dataset using Pandas and scikit-learn. • Standardize the four measurements before PCA. • Reduce the standardized data to two principal components. • Interpret explained variance and principal-axis coefficients. • Visualize species in an interactive Plotly scatter plot and save the results.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads 150 Iris samples and species labels.
	2	DataFrame()	Pandas	Creates tables of components and coefficients.
	3	StandardScaler()	sklearn	Defines mean-zero, unit-variance scaling.
	4	fit_transform()	sklearn	Fits a transformer and transforms the data.
	5	PCA()	sklearn	Defines principal component analysis.
	6	concat()	Pandas	Combines components and original features.
	7	scatter()	Plotly	Creates an interactive 2D scatter plot.
	8	bar()	Plotly	Creates the explained-variance bar chart.
	9	groupby()	Pandas	Groups principal-component scores by species.
	10	mean()/std()	Pandas	Summarizes species centroids and spread.
	11	write_html()/write_image()	Plotly	Exports interactive HTML and static charts.
	12	to_csv()	Pandas	Exports the PCA scores and measurements.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and scikit-learn utilities. 2. Load Iris, rename the four feature columns and map target IDs to species. 3. Inspect sample rows, dataset shape, species counts and descriptive statistics. 4. Standardize the four numeric columns with StandardScaler; verify zero means and unit population standard deviations. 5. Apply PCA(n_components=2, random_state=42) to the standardized matrix. 6. Create a DataFrame of PC1 and PC2 scores, species and original measurements. 7. Inspect explained-variance percentages and principal-axis coefficients (pca.components_.T). 8. Plot PC1 versus PC2 using Plotly, with species colours, symbols and hover data. 9. Fit a four-component PCA to prepare the supporting variance bar chart. 10. Calculate species centroids and standard deviations in principal-component space. 11. Save the interactive scatter plot, static images and PCA CSV; print the analysis summary. Flow: START → Load Iris → Standardize → PCA → 2D scatter / variance chart → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.decomposition import PCA import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(dict(enumerate(iris.target_names)))</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
df.head(10)
# Step 3: Basic verification of the dataset
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
print("\nDescriptive Statistics:")
print(df[feature_columns].describe().round(2))
# Step 4: Standardize the four features (PCA is sensitive to feature
# scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
scaled_df = pd.DataFrame(X_scaled, columns=feature_columns)
print("Mean of each standardized feature (should be ~0):")
print(scaled_df.mean().round(3))
print(
    "\nStandard deviation of each standardized feature (should be ~1):"
)
print(scaled_df.std(ddof=0).round(3))
# Step 5: Apply PCA and keep the first two principal components
pca = PCA(n_components=2, random_state=42)
principal_components = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(
    principal_components, columns=["PC1", "PC2"]
)
pca_df["Species"] = df["Species"]
# keep the original measurements next to the components (used for
# hover information)
pca_df = pd.concat([pca_df, df[feature_columns]], axis=1)
pca_df.head(10)
# Step 6: Explained variance and component loadings
explained = pca.explained_variance_ratio_ * 100
print("Explained Variance Ratio:")
print("PC1 :", round(explained[0], 2), "%")
print("PC2 :", round(explained[1], 2), "%")
print(
    "Total variance captured by PC1 + PC2 :",
    round(explained.sum(), 2),
    "%",
)
loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=feature_columns,
)
print("\nComponent Loadings (contribution of each feature):")
print(loadings.round(3))
# Step 7: Interactive 2D scatter plot of PC1 vs PC2 coloured by
# species
fig = px.scatter(
    pca_df,
    x="PC1",
    y="PC2",
    color="Species",
    symbol="Species",
    hover_data={
        "PC1": ":.3f",
        "PC2": ":.3f",
        "Sepal Length (cm)": True,
        "Sepal Width (cm)": True,
        "Petal Length (cm)": True,
        "Petal Width (cm)": True,
    },
),
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
title="PCA of the Iris Dataset: First Two Principal Components",
labels={
    "PC1": f"PC1 ({explained[0]:.2f}% variance)",
    "PC2": f"PC2 ({explained[1]:.2f}% variance)",
},
)
fig.update_traces(
    marker=dict(
        size=10, opacity=0.85, line=dict(width=1, color="white")
    )
)
fig.update_layout(
    legend_title_text="Species",
    width=900,
    height=550,
    template="plotly_white",
)
fig.show()
# Step 8: Scree plot - variance explained by every component
# (supporting chart)
pca_full = PCA(n_components=4, random_state=42).fit(X_scaled)
scree_df = pd.DataFrame(
    {
        "Principal Component": ["PC1", "PC2", "PC3", "PC4"],
        "Explained Variance (%)": (
            pca_full.explained_variance_ratio_ * 100
        ).round(2),
    }
)
scree_df["Cumulative Variance (%)"] = (
    scree_df["Explained Variance (%)"].cumsum().round(2)
)
print(scree_df)
fig_scree = px.bar(
    scree_df,
    x="Principal Component",
    y="Explained Variance (%)",
    text="Explained Variance (%)",
    title="Scree Plot: Variance Explained by Each Principal Component",
)
fig_scree.update_traces(textposition="outside")
fig_scree.update_layout(
    width=900, height=550, template="plotly_white"
)
fig_scree.show()
# Step 9: Analyse the clusters in the principal-component space
centroids = (
    pca_df.groupby("Species")[["PC1", "PC2"]].mean().round(3)
)
print("Centroid of each species in PC space:")
print(centroids)
spread = pca_df.groupby("Species")[["PC1", "PC2"]].std().round(3)
print(
    "\nSpread (standard deviation) of each species in PC space:"
)
print(spread)
# Step 10: Save the interactive chart, a static image and the PCA
# dataset
fig.write_html("program16_pca_scatter.html")
print(
    "Interactive PCA scatter plot saved as program16_pca_scatter.html"
)
try:
    fig.write_image(
        "program16_pca_scatter.png",
        width=900,
        height=550,
        scale=2,
    )
)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
fig_scree.write_image(  
    "program16_scree_plot.png", width=900, height=550, scale=2  
)  
print(  
    "Static images saved as program16_pca_scatter.png and program16_scree_plot.png"  
)  
except Exception as error:  
    print(  
        "Static image not saved (kaleido not available):", error  
    )  
pca_df.to_csv("program16_iris_pca.csv", index=False)  
print("PCA dataset saved as program16_iris_pca.csv")  
# Step 11: Summary of the analysis  
print("PCA ANALYSIS SUMMARY")  
print("-" * 45)  
print("Total samples           :", len(pca_df))  
print("Original features         :", len(feature_columns))  
print("Components retained       : 2")  
print("Variance explained by PC1  :", round(explained[0], 2), "%")  
print("Variance explained by PC2  :", round(explained[1], 2), "%")  
print(  
    "Total variance retained   :", round(explained.sum(), 2), "%"  
)  
print(  
    "Most influential feature  :",  
    loadings["PC1"].abs().idxmax(),  
    "(on PC1)",  
)  
print("Well separated species    : setosa")  
print("Partially overlapping     : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

**Output
Snapshots
With
Explanation/
Observations**

```
Dataset Shape: (150, 5)

Species Counts:
Species
setosa      50
versicolor 50
virginica   50
Name: count, dtype: int64

Descriptive Statistics:
      Sepal Length (cm)  Sepal Width (cm)  Petal Length (cm)  Petal Width (cm)
count              150.00              150.00              150.00              150.00
mean                5.84                3.06                3.76                1.20
std                 0.83                0.44                1.77                0.76
min                 4.30                2.00                1.00                0.10
25%                 5.10                2.80                1.60                0.30
50%                 5.80                3.00                4.35                1.30
75%                 6.40                3.30                5.10                1.80
max                 7.90                4.40                6.90                2.50

Mean of each standardized feature (should be ~0):
Sepal Length (cm)  -0.0
Sepal Width (cm)   -0.0
Petal Length (cm)  -0.0
Petal Width (cm)   -0.0
dtype: float64

Standard deviation of each standardized feature (should be ~1):
Sepal Length (cm)   1.0
Sepal Width (cm)    1.0
Petal Length (cm)   1.0
Petal Width (cm)    1.0
dtype: float64
```

Fig. 1.1: Dataset verification and feature standardization

The dataset contains 150 records and five DataFrame columns: four measurements and one species label. Each species has 50 observations. The transformed feature matrix has zero means (up to floating-point rounding) and population standard deviations of one.

```
Explained Variance Ratio:
PC1 : 72.96 %
PC2 : 22.85 %
Total variance captured by PC1 + PC2 : 95.81 %

Component Loadings (contribution of each feature):
              PC1    PC2
Sepal Length (cm)  0.521  0.377
Sepal Width (cm)   -0.269  0.923
Petal Length (cm)  0.580  0.024
Petal Width (cm)   0.565  0.067
```

Fig. 1.2: Explained variance and principal-axis coefficients

PC1 and PC2 retain 95.81% of the variance. Petal length has the largest absolute PC1 coefficient (0.580). The notebook calls these values loadings; here they are the principal-axis coefficients from `pca.components_.T`, not correlation loadings.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

PCA of the Iris Dataset: First Two Principal Components

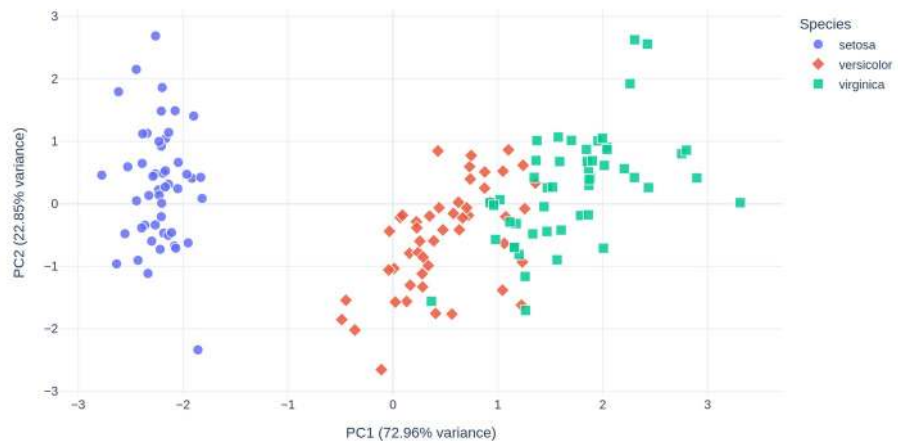


Fig. 1.3: Plotly scatter plot of the first two principal components

Setosa occupies the negative PC1 region and is separated from the other species. Versicolor and virginica overlap partly. Colours and symbols represent known species, not clusters generated by PCA.

Scree Plot: Variance Explained by Each Principal Component

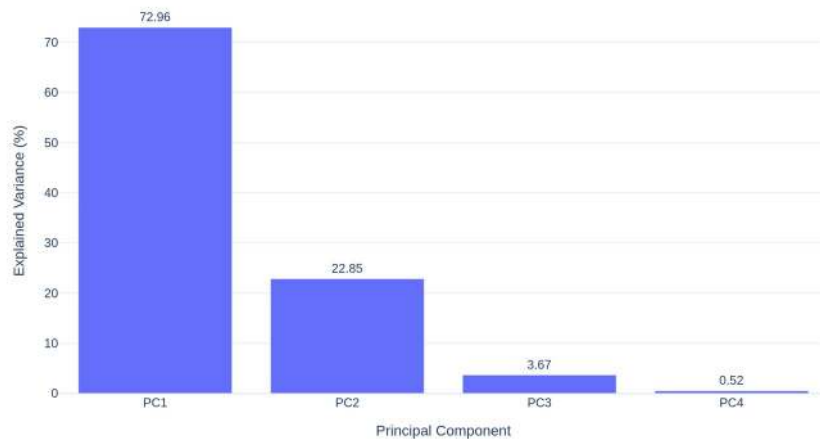


Fig. 1.4: Variance explained by all four principal components

The four variance percentages are 72.96%, 22.85%, 3.67% and 0.52%. Retaining the first two components loses approximately 4.19% of the standardized variance.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

	Centroid of each species in PC space: <table><thead><tr><th></th><th>PC1</th><th>PC2</th></tr></thead><tbody><tr><td>Species</td><td></td><td></td></tr><tr><td>setosa</td><td>-2.225</td><td>0.289</td></tr><tr><td>versicolor</td><td>0.496</td><td>-0.550</td></tr><tr><td>virginica</td><td>1.728</td><td>0.261</td></tr></tbody></table> Spread (standard deviation) of each species in PC space: <table><thead><tr><th></th><th>PC1</th><th>PC2</th></tr></thead><tbody><tr><td>Species</td><td></td><td></td></tr><tr><td>setosa</td><td>0.206</td><td>0.934</td></tr><tr><td>versicolor</td><td>0.464</td><td>0.833</td></tr><tr><td>virginica</td><td>0.571</td><td>0.876</td></tr></tbody></table> Fig. 1.5: Species centroids and spread in principal-component space Mean PC1 scores increase from setosa (−2.225) through versicolor (0.496) to virginica (1.728). Species labels are used only after projection for these summaries. The reported spreads are sample standard deviations.		PC1	PC2	Species			setosa	-2.225	0.289	versicolor	0.496	-0.550	virginica	1.728	0.261		PC1	PC2	Species			setosa	0.206	0.934	versicolor	0.464	0.833	virginica	0.571	0.876
	PC1	PC2																													
Species																															
setosa	-2.225	0.289																													
versicolor	0.496	-0.550																													
virginica	1.728	0.261																													
	PC1	PC2																													
Species																															
setosa	0.206	0.934																													
versicolor	0.464	0.833																													
virginica	0.571	0.876																													
	Interactive PCA scatter plot saved as program16_pca_scatter.html Static images saved as program16_pca_scatter.png and program16_scree_plot.png PCA dataset saved as program16_iris_pca.csv PCA ANALYSIS SUMMARY <pre>----- Total samples : 150 Original features : 4 Components retained : 2 Variance explained by PC1 : 72.96 % Variance explained by PC2 : 22.85 % Total variance retained : 95.81 % Most influential feature : Petal Length (cm) (on PC1) Well separated species : setosa Partially overlapping : versicolor and virginica</pre> Fig. 1.6: File export confirmation and PCA summary Execution produced the interactive scatter HTML, two static charts and a CSV containing PC1, PC2, species and the four original measurements. The summary confirms 150 observations and 95.81% retained variance.																														
Conclusions	The Iris dataset was successfully standardized and reduced from four measurements to two principal components. PC1 explains 72.96% and PC2 explains 22.85% of the standardized variance, retaining 95.81% in total. Setosa is clearly separated in the projection, while versicolor and virginica partly overlap. PCA is an unsupervised dimensionality-reduction method, not a clustering algorithm; species colours are reference labels. Plotly provides an interactive view, and the saved HTML and CSV support further exploration.																														

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

References

1. scikit-learn PCA documentation:
2. scikit-learn Iris dataset / StandardScaler documentation:
3. Plotly Python documentation:
4. Pandas documentation:
5. Python notebook (GitHub link): [Open Program 16 on GitHub](#)
6. Class lecture link: [Open Week 8 lecture](#)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly.			
Objectives	• Prepare and standardize the four Iris measurements. • Compare inertia and silhouette scores for candidate cluster counts. • Fit K-means with three clusters and inspect their composition. • Evaluate clusters using silhouette score and adjusted Rand index. • Visualize clusters and their centres in an interactive Plotly 3D scatter plot.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads the Iris dataset and labels.
	2	StandardScaler()	sklearn	Standardizes all four numeric features.
	3	KMeans()	sklearn	Defines the K-means clustering model.
	4	fit_predict()	sklearn	Fits the model and returns cluster labels.
	5	silhouette_score()	sklearn	Measures cluster cohesion and separation.
	6	adjusted_rand_score()	sklearn	Compares clusters with known species labels.
	7	crosstab()	Pandas	Tabulates cluster and species membership.
	8	inverse_transform()	sklearn	Converts centres back to original cm units.
	9	line()	Plotly	Plots inertia for the candidate k values.
	10	scatter_3d()/Scatter3d()	Plotly	Plots observations and cluster centres in 3D.
	11	write_html()/write_image()	Plotly	Saves interactive HTML and static charts.
	12	to_csv()	Pandas	Saves original features and cluster labels.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and the scikit-learn clustering/evaluation utilities. 2. Load Iris, rename the feature columns and map the known species labels. 3. Standardize only the four measurements using StandardScaler. 4. Fit K-means for $k = 1$ to 10; collect inertia and silhouette scores (silhouette is undefined for $k = 1$). 5. Plot inertia versus k. Use $k = 3$ to compare with the three known species; do not claim it maximizes the silhouette score. 6. Fit KMeans($n_clusters=3$, $init="k-means++"$, $n_init=10$, $random_state=42$) and assign cluster labels. 7. Compute cluster sizes, a cluster/species cross-tabulation, silhouette score and adjusted Rand index. 8. Convert the cluster centres from standardized units back to cm. 9. Plot sepal length, sepal width and petal length in 3D; use colour for cluster, symbol for species and hover data for petal width. 10. Overlay cluster centres, save the interactive HTML, static charts and clustered CSV, and print the summary. Flow: START → Load Iris → Standardize → Compare k → Fit K-means → Evaluate → 3D scatter → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.graph_objects as go import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.cluster import KMeans from sklearn.metrics import silhouette_score, adjusted_rand_score import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
dict(enumerate(iris.target_names))
)
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
df.head(10)
# Step 3: Standardize the features so that every measurement has equal
# weight in the distance calculation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
print("Scaled feature matrix shape:", X_scaled.shape)
print("First 5 scaled rows:")
print(np.round(X_scaled[:5], 3))
# Step 4: Elbow method and silhouette score to choose the number of
# clusters (k)
k_values = range(1, 11)
inertia = []
silhouette = []
for k in k_values:
    model = KMeans(
        n_clusters=k, init="k-means++", n_init=10, random_state=42
    )
    labels = model.fit_predict(X_scaled)
    inertia.append(model.inertia_)
    silhouette.append(
        silhouette_score(X_scaled, labels) if k > 1 else np.nan
    )
elbow_df = pd.DataFrame(
    {
        "k": list(k_values),
        "Inertia (WCSS)": np.round(inertia, 3),
        "Silhouette Score": np.round(silhouette, 3),
    }
)
print(elbow_df.to_string(index=False))
fig_elbow = px.line(
    elbow_df,
    x="k",
    y="Inertia (WCSS)",
    markers=True,
    title="Elbow Method: Inertia vs Number of Clusters (k)",
)
fig_elbow.update_layout(
    width=900,
    height=550,
    template="plotly_white",
    xaxis=dict(dtick=1),
)
fig_elbow.show()
# Step 5: Fit K-means with k = 3 (three Iris species) and assign
# cluster labels
kmeans = KMeans(
    n_clusters=3, init="k-means++", n_init=10, random_state=42
)
df["Cluster"] = kmeans.fit_predict(X_scaled)
df["Cluster"] = "Cluster " + df["Cluster"].astype(str)
print("Number of iterations to converge:", kmeans.n_iter_)
print("Final inertia (WCSS):", round(kmeans.inertia_, 3))
print("\nCluster Sizes:")
print(df["Cluster"].value_counts().sort_index())
df.head(10)
# Step 6: Compare the discovered clusters with the actual species
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
comparison = pd.crosstab(df["Cluster"], df["Species"])
print("Cluster vs Species Cross-tabulation:")
print(comparison)
sil = silhouette_score(X_scaled, kmeans.labels_)
ari = adjusted_rand_score(iris.target, kmeans.labels_)
print("\nSilhouette Score (k = 3) : ", round(sil, 3))
print("Adjusted Rand Index vs species:", round(ari, 3))
# percentage of points whose cluster matches the dominant species of
# that cluster
correctly_grouped = comparison.max(axis=1).sum()
print(
    "Points grouped with their dominant species:",
    correctly_grouped,
    "of",
    len(df),
    "(",
    round(correctly_grouped / len(df) * 100, 2),
    "% )",
)
# Step 7: Cluster centres converted back to the original units (cm)
centers_cm = scaler.inverse_transform(kmeans.cluster_centers_)
centers_df = pd.DataFrame(
    centers_cm, columns=feature_columns
).round(3)
centers_df.insert(
    0, "Cluster", ["Cluster 0", "Cluster 1", "Cluster 2"]
)
print("Cluster Centres (original units):")
print(centers_df.to_string(index=False))
# Step 8: Interactive 3D scatter plot of the clusters (with cluster
# centres marked)
fig = px.scatter_3d(
    df.sort_values("Cluster"),
    x="Sepal Length (cm)",
    y="Sepal Width (cm)",
    z="Petal Length (cm)",
    color="Cluster",
    symbol="Species",
    hover_data={"Petal Width (cm)": True},
    title="K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View",
    opacity=0.8,
)
fig.update_traces(marker=dict(size=4))
# add the three cluster centres as large black diamonds
fig.add_trace(
    go.Scatter3d(
        x=centers_df["Sepal Length (cm)"],
        y=centers_df["Sepal Width (cm)"],
        z=centers_df["Petal Length (cm)"],
        mode="markers",
        marker=dict(size=8, color="black", symbol="diamond"),
        hovertext=centers_df["Cluster"] + " centre",
        name="Cluster Centres",
    )
)
fig.update_layout(
    width=900,
    height=650,
    legend_title_text="Cluster / Species",
    margin=dict(l=0, r=0, b=0, t=60),
    scene=dict(
        xaxis_title="Sepal Length (cm)",
        yaxis_title="Sepal Width (cm)",
        zaxis_title="Petal Length (cm)",
        aspectmode="cube",
        camera=dict(eye=dict(x=1.7, y=1.5, z=0.9)),
    ),
    template="plotly_white",
)
fig.show()
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
# Step 9: Save the interactive chart, static images and the clustered
# dataset
fig.write_html("program17_kmeans_3d_scatter.html")
print(
    "Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html"
)
try:
    fig.write_image(
        "program17_kmeans_3d_scatter.png",
        width=900,
        height=650,
        scale=2,
    )
    fig_elbow.write_image(
        "program17_elbow_plot.png", width=900, height=550, scale=2
    )
    print(
        "Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png"
    )
except Exception as error:
    print(
        "Static image not saved (kaleido not available):", error
    )
df.to_csv("program17_iris_kmeans_clusters.csv", index=False)
print(
    "Clustered dataset saved as program17_iris_kmeans_clusters.csv"
)
# Step 10: Summary of the analysis
print("K-MEANS CLUSTERING SUMMARY")
print("-" * 45)
print("Total samples           :", len(df))
print(
    "Features used             :",
    len(feature_columns),
    "(standardized)",
)
print("Number of clusters (k)    : 3")
print(
    "Cluster sizes              :",
    df["Cluster"].value_counts().sort_index().tolist(),
)
print("Silhouette score           :", round(sil, 3))
print("Adjusted Rand Index        :", round(ari, 3))
print(
    "Agreement with species     :",
    round(correctly_grouped / len(df) * 100, 2),
    "%",
)
print("Perfectly separated species : setosa")
print("Overlapping species        : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

**Output
Snapshots
With
Explanation/
Observations**

Dataset Shape: (150, 5)

Species Counts:

Species

setosa 50

versicolor 50

virginica 50

Name: count, dtype: int64

Scaled feature matrix shape: (150, 4)

First 5 scaled rows:

```
[[ -0.901  1.019 -1.34  -1.315]
```

```
 [ -1.143 -0.132 -1.34  -1.315]
```

```
 [ -1.385  0.328 -1.397 -1.315]
```

```
 [ -1.507  0.098 -1.283 -1.315]
```

```
 [ -1.022  1.249 -1.34  -1.315]]
```

Fig. 2.1: Dataset and standardized feature matrix

All 150 observations and four numeric features are used for clustering. Species labels are kept separately for plotting and evaluation; they are not included in the distance calculations.

k	Inertia (WCSS)	Silhouette Score
1	600.000	NaN
2	222.362	0.582
3	139.820	0.460
4	114.093	0.387
5	90.928	0.346
6	81.544	0.317
7	72.631	0.320
8	62.541	0.339
9	55.119	0.342
10	47.391	0.352

Fig. 2.2: Inertia and silhouette scores for k = 1 to 10

Inertia decreases as more clusters are added. The highest silhouette score among the tested values is at k = 2 (0.582), not k = 3 (0.460). The notebook uses k = 3 for comparison with the three known Iris species. A silhouette score is undefined for a single cluster.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Elbow Method: Inertia vs Number of Clusters (k)

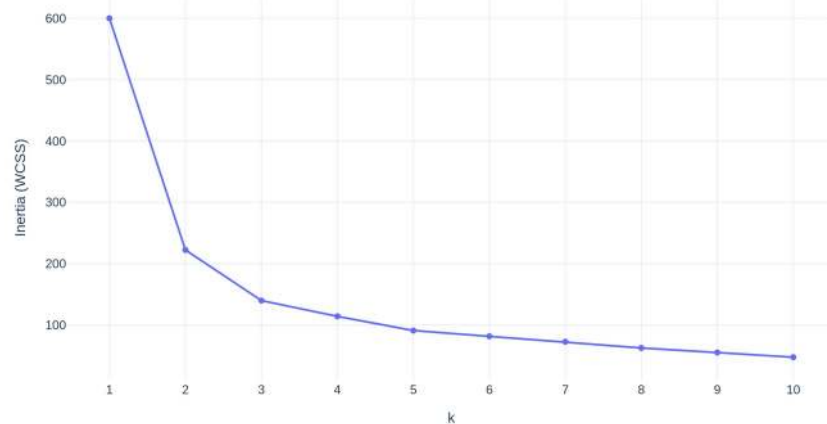


Fig. 2.3: Plotly elbow chart

The large early reductions in inertia are followed by progressively smaller improvements. This chart supports exploring a small number of clusters but does not prove that three is uniquely optimal.

Number of iterations to converge: 4
Final inertia (WCSS): 139.82

Cluster Sizes:

Cluster

Cluster 0 53

Cluster 1 50

Cluster 2 47

Name: count, dtype: int64

Cluster vs Species Cross-tabulation:

Species	setosa	versicolor	virginica
Cluster 0	0	39	14
Cluster 1	50	0	0
Cluster 2	0	11	36

Silhouette Score (k = 3) : 0.46

Adjusted Rand Index vs species: 0.62

Points grouped with their dominant species: 125 of 150 (83.33 %)

Fig. 2.4: Cluster sizes, species cross-tabulation and evaluation

Cluster 1 contains all 50 setosa samples. Cluster 0 contains 39 versicolor and 14 virginica; Cluster 2 contains 11 versicolor and 36 virginica. Cluster IDs are arbitrary. The dominant-species counts are 39, 50 and 36 for Clusters 0, 1 and 2, respectively: 125/150 (83.33%) in total.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Cluster Centres (original units):

Cluster	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)
Cluster 0	5.802	2.674	4.370	1.413
Cluster 1	5.006	3.428	1.462	0.246
Cluster 2	6.781	3.096	5.511	1.972

Fig. 2.5: Cluster centres in centimetres

Inverse transformation makes each centre interpretable in original measurement units. All four dimensions contribute to model fitting, even though only three dimensions can be displayed in the scatter plot.

K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View

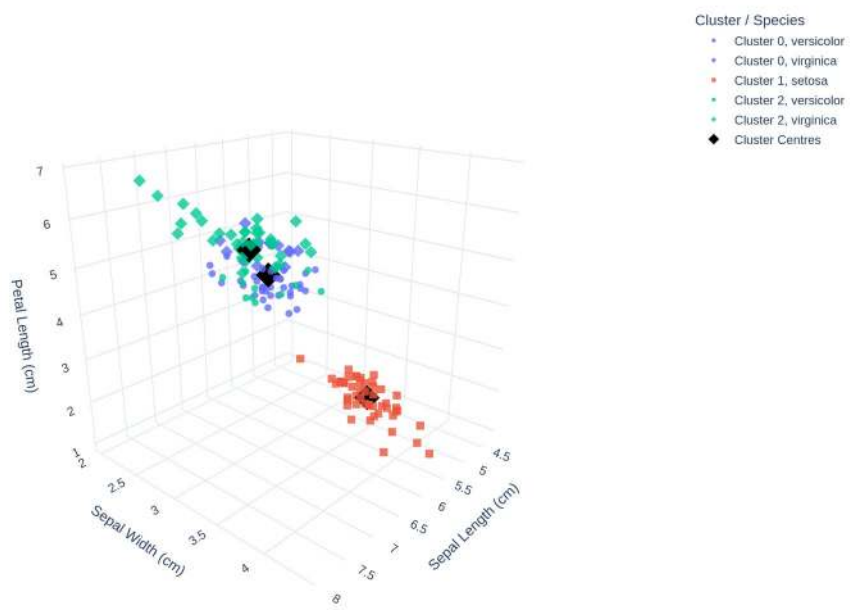


Fig. 2.6: Interactive 3D cluster plot with centres

The static view shows sepal length, sepal width and petal length. Colour encodes cluster and symbol encodes species. Black diamonds mark cluster centres. Petal width is available on hover in the interactive HTML.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

	Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png Clustered dataset saved as program17_iris_kmeans_clusters.csv K-MEANS CLUSTERING SUMMARY <pre> ----- Total samples : 150 Features used : 4 (standardized) Number of clusters (k) : 3 Cluster sizes : [53, 50, 47] Silhouette score : 0.46 Adjusted Rand Index : 0.62 Agreement with species : 83.33 % Perfectly separated species : setosa Overlapping species : versicolor and virginica </pre> Fig. 2.7: File export confirmation and K-means summary Execution converged in four iterations with inertia 139.820. It saved the interactive 3D scatter, two static charts and the clustered dataset. The output confirms cluster sizes 53, 50 and 47, silhouette 0.460 and adjusted Rand index 0.620.
Conclusions	K-means successfully grouped 150 standardized Iris observations into three clusters of sizes 53, 50 and 47. The final inertia is 139.820, the silhouette score is 0.460 and the adjusted Rand index is 0.620. All 50 setosa samples form one cluster; versicolor and virginica overlap. Dominant-species agreement is 125/150 (83.33%), a descriptive cluster-purity measure rather than supervised test accuracy. The 3D Plotly chart displays three original measurements, while clustering uses all four standardized features. The result demonstrates useful but imperfect agreement between unsupervised groups and biological species.
References	1. scikit-learn K-means documentation: 2. scikit-learn Iris dataset / StandardScaler documentation: 3. Plotly Python documentation: 4. Pandas documentation: 5. Python notebook (GitHub link): Open Program 17 on GitHub 6. Class lecture link: Open Week 8 lecture